MAANG-Focused LeetCode DSA Preparation Guide

A pattern-first, ROI-driven roadmap from fundamentals to MAANG-level interviews.

How To Use This Document

1. Read the **Pattern Card** at the top of each topic before touching problems — recognition signals matter more than memorizing solutions.
2. Work top-to-bottom within a topic (🟢 → 🔵 → 🟠 → 🔴); difficulty is ordered for *learning*, not strictly by LeetCode's own difficulty tag.
3. Use the **Priority** stars to triage when time is short — ⭐ ⭐ ⭐ ⭐ ⭐ problems are non-negotiable.
4. Follow one of the three **Learning Tracks** (Foundation / Placement / MAANG) based on your timeline.
5. Use the **Revision System** at the end for spaced repetition, the **Mistake Log**, and **Interview Simulation Sets**.

Legend: 🟢 Foundation · 🔵 Pattern Building · 🟠 Interview Level · 🔴 MAANG Level **Priority:** ⭐ ⭐ ⭐ ⭐ ⭐ Must Do · ⭐ ⭐ ⭐ ⭐ Highly Recommended · ⭐ ⭐ ⭐ Recommended · ⭐ ⭐ Optional · ⭐ Advanced/Niche

---

Table of Contents

1. Arrays & Hashing
2. Two Pointers
3. Sliding Window
4. Binary Search
5. Linked Lists
6. Stack & Monotonic Stack
7. Queue & Monotonic Queue
8. Recursion & Backtracking
9. Trees
10. Heap / Priority Queue
11. Graphs
12. Dynamic Programming
13. Greedy
14. Intervals
15. Bit Manipulation
16. Math & Number Theory
17. Tries
18. Advanced Data Structures
19. Learning Tracks (A/B/C)

20. Revision System (50/100 lists, Pattern Sheet, Mistake Log, Interview Simulations)
21. 12–16 Week Roadmap & Daily Workflow

---

1. Arrays & Hashing

Pattern Card

- **Recognition signals:** "contains duplicate", "find pair/triplet summing to X", "group by property", "count frequency", "subarray sum equals K", "longest sequence".
- **Core idea:** Trade space for time — a hash map/set turns $O(n^2)$ lookups into $O(1)$; prefix sums turn "range sum" questions into $O(1)$ queries after $O(n)$ preprocessing.
- **Typical complexity:** $O(n)$ time, $O(n)$ space for hashing; $O(n)$ time $O(1)$ space for Kadane-style scans.
- **Common traps:** Forgetting to handle duplicates/negatives in prefix sums; using a map when sorting + two pointers would avoid extra space; off-by-one errors in prefix-sum ranges.
- **Problems to practice:** see table below.

#	Problem	Difficulty	Level	Priority
1	Contains Duplicate	Easy	🟢	⭐⭐⭐⭐⭐
2	Valid Anagram	Easy	🟢	⭐⭐⭐⭐
3	Two Sum	Easy	🟢	⭐⭐⭐⭐⭐
4	Group Anagrams	Medium	🔵	⭐⭐⭐⭐⭐
5	Top K Frequent Elements	Medium	🔵	⭐⭐⭐⭐⭐
6	Product of Array Except Self	Medium	🔵	⭐⭐⭐⭐⭐
7	Valid Sudoku	Medium	🔵	⭐⭐⭐
8	Longest Consecutive Sequence	Medium	🟠	⭐⭐⭐⭐⭐
9	Subarray Sum Equals K	Medium	🟠	⭐⭐⭐⭐⭐
10	Continuous Subarray Sum	Medium	🟠	⭐⭐⭐
11	Maximum Subarray (Kadane's)	Medium	🔵	⭐⭐⭐⭐⭐
12	Maximum Product Subarray	Medium	🟠	⭐⭐⭐⭐
13	Range Sum Query - Immutable	Easy	🟢	⭐⭐⭐
14	Subarray Sums Divisible by K	Medium	🟠	⭐⭐⭐
15	First Missing Positive	Hard	🔴	⭐⭐⭐⭐
16	Insert Delete GetRandom O(1)	Medium	🟠	⭐⭐⭐

Problem Details

#1. Contains Duplicate — Easy · 🟢 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Set membership. Key Concept: use a hash set to detect repeats in one pass. Why it matters: the simplest possible "trade space for time" problem; sets the mental model for everything else in this section. Prerequisites: none. Approach: iterate, insert into set, return true if already present. Insight: sorting also works in O(n log n) with O(1) space — know both trade-offs. Common mistake: using nested loops ($O(n^2)$) under time pressure. Variation: "contains duplicate within distance k" (sliding window + set). Companies: Amazon, Google.

#2. Valid Anagram — Easy · 🟢 · ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Frequency map. Key Concept: character frequency counting. Why it matters: frequency-map pattern reused constantly in string problems. Prerequisites: #1. Approach: count chars in one string, decrement using the other, check all zero. Insight: sorting both strings and comparing is a simpler O(n log n) alternative. Common mistake: ignoring Unicode/case sensitivity assumptions. Variation: Group Anagrams (#4). Companies: Amazon, Bloomberg.

#3. Two Sum — Easy · 🟢 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Complement lookup. Key Concept: store value→index while scanning, check for complement. Why it matters: the canonical interview warm-up; complement-lookup generalizes to 3Sum, 4Sum, subarray-sum problems. Prerequisites: none. Approach: single pass hash map of seen values to index. Insight: only need one pass because you check-then-insert. Common mistake: using the same element twice; returning values instead of indices. Follow-up: Two Sum II (sorted array → two pointers), Two Sum BST. Companies: Amazon, Google, Meta, Apple, Microsoft — the most frequently asked question industry-wide.

#4. Group Anagrams — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Canonical key. Key Concept: map each string to a canonical signature (sorted string or 26-count tuple) and bucket by it. Why it matters: teaches the "canonical key" trick used across many grouping problems. Prerequisites: #2. Approach: hash map of signature → list of strings. Insight: counting-based signature (O(n)) beats sorting-based signature (O(n log n)) for large inputs. Common mistake: using sorted string as key when a count-array key would be faster. Variation: Group Shifted Strings. Companies: Amazon, Uber, Meta.

#5. Top K Frequent Elements — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing + Heap / Bucket sort. Key Concept: count frequencies, then either a heap of size K or bucket sort by frequency. Why it matters: bridges Arrays & Hashing into the Heap topic; bucket sort gives O(n) which impresses interviewers. Prerequisites: #1, #2. Approach: frequency map → bucket array indexed by frequency → read off top K. Insight: bucket sort avoids the O(n log k) heap cost entirely. Common mistake: defaulting to sort-everything O(n log n) without mentioning the O(n) bucket approach. Variation: Top K Frequent Words (tie-break lexicographically). Companies: Amazon, Meta, Yelp.

#6. Product of Array Except Self — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Prefix/Suffix products. Key Concept: precompute prefix product and suffix product without division. Why it matters: the classic "no division allowed" constraint forces the prefix/suffix pattern used broadly in DP and array problems. Prerequisites: none. Approach: left-to-right prefix array, right-to-left suffix pass multiplied into the same output array for O(1) extra space. Insight: you can

do it in a single output array reused as both prefix and suffix storage. Common mistake: using division and not handling zeros correctly. Variation: same with a zero in the array. Companies: Amazon, Meta, Apple, Microsoft.

#7. Valid Sudoku — Medium · 🔵 · ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Matrix traversal. Key Concept: encode (row, col, box) constraints as hash set keys. Why it matters: teaches multi-dimensional constraint hashing seen again in matrix problems. Prerequisites: #1. Approach: three sets of sets (rows, cols, boxes) keyed by index, encode box index as (r/3, c/3). Insight: a single set with composite keys like `"r5#3"` avoids needing three separate structures. Common mistake: forgetting box index formula. Variation: Sudoku Solver (backtracking). Companies: Apple, Uber.

#8. Longest Consecutive Sequence — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing / Sequence detection. Key Concept: only start counting a sequence from its true starting number (num-1 not in set). Why it matters: teaches the "skip unless start of run" trick that appears in interval/graph problems too. Prerequisites: #1. Approach: put all numbers in a set, for each number check if num-1 absent, then count forward. Insight: this achieves O(n) despite looking like nested loops, because inner loops only execute for true sequence starts. Common mistake: not checking the "is this a start" condition, causing $O(n^2)$. Variation: longest consecutive sequence in a binary tree. Companies: Google, Meta, Pinterest.

#9. Subarray Sum Equals K — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Prefix sum + Hashing. Key Concept: store frequency of prefix sums seen so far; subarray sum = prefixSum[j] - prefixSum[i]. Why it matters: the gateway problem for the entire "prefix sum + hash map" family (divisible by K, equal 0/1 count, etc.). Prerequisites: #6. Approach: running prefix sum, look up `prefixSum - k` in a frequency map, increment count. Insight: initialize map with {0:1} to count subarrays starting at index 0. Common mistake: forgetting the {0:1} seed value, undercounting valid subarrays. Follow-up: Subarray Sums Divisible by K (#14), Contiguous Array (equal 0s/1s). Companies: Google, Meta, Amazon.

#10. Continuous Subarray Sum — Medium · 🔴 · ⭐ ⭐ ⭐ Primary/Secondary Pattern: Prefix sum + Modulo hashing. Key Concept: two indices with the same prefix-sum-mod-k imply the subarray between them is divisible by k. Why it matters: reinforces modular-arithmetic + prefix-sum combination. Prerequisites: #9. Approach: store first-seen index of each remainder; check for gap ⩾ 2. Insight: store the *index*, not the count, since only the earliest occurrence maximizes subarray length. Common mistake: mishandling the length ⩾ 2 requirement and negative-mod languages. Variation: Subarray Sums Divisible by K. Companies: Google.

#11. Maximum Subarray (Kadane's) — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Dynamic Programming (1D) / Greedy. Key Concept: at each index decide whether to extend the previous subarray or start fresh. Why it matters: Kadane's is the entry point to 1D DP thinking — "extend vs restart" recurs throughout DP. Prerequisites: none. Approach: `cur = max(num, cur+num)`, track global max. Insight: the moment cur drops below 0 it can never help a future sum. Common mistake: initializing global max to 0 instead of `-infinity` (fails on all-negative arrays). Follow-up: return the subarray itself; Maximum Circular Subarray. Companies: Amazon, Microsoft, LinkedIn — extremely common.

#12. Maximum Product Subarray — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Dynamic Programming (1D). Key Concept: track both running max AND running min, since a

negative number can flip min into max. Why it matters: teaches "track two DP states" when a single running value isn't sufficient. Prerequisites: #11. Approach: at each step compute candidates (num, maxSoFar*num, minSoFar*num); new max/min are the extremes of those three. Insight: swap max/min trackers whenever num is negative, before updating (or compute both, then pick). Common mistake: only tracking max and losing the sign-flip case. Variation: max product of two/three numbers (no DP). Companies: Amazon, LinkedIn.

#13. Range Sum Query - Immutable — Easy · 🟢 · ⭐ ⭐ ⭐ Primary/Secondary Pattern: Prefix sum. Key Concept: precompute prefix sums once, answer queries in O(1). Why it matters: the textbook intro to prefix sums before jumping to 2D or DP contexts. Prerequisites: none. Approach: `prefix[i] = prefix[i-1] + nums[i-1]`; range sum = `prefix[r+1]-prefix[l]`. Insight: pad prefix array with a leading 0 to avoid edge-case branching. Common mistake: off-by-one in inclusive ranges. Follow-up: Range Sum Query 2D, Range Sum Query Mutable (Fenwick Tree). Companies: Google.

#14. Subarray Sums Divisible by K — Medium · 🟠 · ⭐ ⭐ ⭐ Primary/Secondary Pattern: Prefix sum + Modulo hashing. Key Concept: identical to #9 but keyed by remainder instead of raw sum. Why it matters: cements the prefix-sum-hashing generalization (works for equality, modulo, or any invariant). Prerequisites: #9, #10. Approach: hash map of remainder frequency, handle negative remainders with `((r % k) + k) % k`. Insight: negative modulo handling is the main gotcha across languages. Common mistake: not normalizing negative remainders. Variation: none beyond #9/#10. Companies: Google, Amazon.

#15. First Missing Positive — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ Primary/Secondary Pattern: Array as Hash Table (in-place). Key Concept: place each value `v` at index `v-1` via swapping, then scan for the first mismatch. Why it matters: the flagship "use the array itself as a hash set" trick for O(1) space, a favorite MAANG hard. Prerequisites: #1. Approach: cyclic sort — swap `nums[i]` to `nums[nums[i]-1]` while in range and not already correct; then find first index where `nums[i] != i+1`. Insight: answer is always in `[1, n+1]`, bounding the search space. Common mistake: infinite loop from mishandling duplicate/self-swaps. Variation: Find All Duplicates in an Array (same cyclic-sort trick). Companies: Amazon, Microsoft, Bloomberg.

#16. Insert Delete GetRandom O(1) — Medium · 🟠 · ⭐ ⭐ ⭐ Primary/Secondary Pattern: Hashing + Array design. Key Concept: combine a hash map (value→index) with a dynamic array to get O(1) insert/delete/random-access. Why it matters: a design problem that tests whether hashing intuition extends beyond lookups to full data-structure design. Prerequisites: #1. Approach: on delete, swap the target with the last array element before popping, and update the map. Insight: "swap-with-last-then-pop" avoids O(n) shifting on delete. Common mistake: forgetting to update the map index for the swapped element. Variation: Insert Delete GetRandom O(1) - Duplicates allowed. Companies: Meta, Uber.

---

2. Two Pointers

Pattern Card

- **Recognition signals:** sorted array, "pair/triplet summing to target", palindrome check, partitioning, "remove duplicates in place", merging two sorted structures.

- **Core idea:** two indices moving toward each other (opposite-direction) or in the same direction at different speeds shrink the search space from $O(n^2)$ to $O(n)$, usually after sorting.
- **Typical complexity:** $O(n \log n)$ if sorting is required, else $O(n)$; $O(1)$ extra space.
- **Common traps:** forgetting to skip duplicates in triplet problems; moving the wrong pointer when the condition fails; off-by-one on the "meet in the middle" stop condition.

#	Problem	Difficulty	Level	Priority
17	Valid Palindrome	Easy	🟢	⭐⭐⭐⭐
18	Two Sum II - Input Array Is Sorted	Medium	🟢	⭐⭐⭐⭐⭐
19	3Sum	Medium	🔵	⭐⭐⭐⭐⭐
20	3Sum Closest	Medium	🔵	⭐⭐⭐
21	Container With Most Water	Medium	🔵	⭐⭐⭐⭐⭐
22	Trapping Rain Water	Hard	🔴	⭐⭐⭐⭐⭐
23	Sort Colors	Medium	🔵	⭐⭐⭐⭐
24	Remove Duplicates from Sorted Array	Easy	🟢	⭐⭐⭐
25	Boats to Save People	Medium	🟠	⭐⭐⭐

Problem Details

#17. Valid Palindrome — Easy · 🟢 · ⭐⭐⭐⭐: Opposite-direction pointers skipping non-alphanumerics. Key concept: in-place two-ended scan. Matters because it's the simplest two-pointer template. Prereq: none. Approach: left/right indices, skip invalid chars, compare case-insensitively. Insight: no need to build a cleaned string first — filter on the fly. Mistake: forgetting `toLowerCase`. Variation: Valid Palindrome II (delete at most one char). Companies: Meta, Amazon.

#18. Two Sum II — Medium · 🟢 · ⭐⭐⭐⭐⭐: Opposite-direction pointers on sorted input. Key concept: move left pointer up if sum too small, right pointer down if too large. Matters as the template for 3Sum/4Sum. Prereq: #3. Approach: two pointers from both ends. Insight: sortedness is what makes monotonic pointer movement correct. Mistake: not realizing input is 1-indexed in LeetCode's version. Variation: 3Sum, 4Sum. Companies: Amazon, Google.

#19. 3Sum — Medium · 🔵 · ⭐⭐⭐⭐⭐: Sort + fix one element + two pointers on the rest. Key concept: reduce 3-variable search to 2-variable via sorting and fixing. Matters as one of the most-asked MAANG mediums, and the template for k-Sum generalization. Prereq: #18. Approach: sort array; for each i, two-pointer search for pairs summing to `-nums[i]`; skip duplicate i and duplicate pair values. Insight: the "skip duplicates" logic must run on both the outer loop and the inner pointers. Mistake: producing duplicate triplets. Variation: 4Sum, 3Sum Smaller, k-Sum general recursion. Companies: Meta, Amazon, Microsoft, Google — top-asked question.

#20. 3Sum Closest — Medium · 🔵 · ⭐ ⭐ ⭐: Same skeleton as 3Sum but track closest sum instead of exact matches. Matters for showing you can adapt a memorized template to a variant constraint. Prereq: #19. Approach: sort, fix i, two-pointer while tracking min |sum-target|. Insight: no duplicate-skipping needed since we want closest, not unique sets. Mistake: initializing closest sum incorrectly. Companies: Meta.

#21. Container With Most Water — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Opposite-direction pointers with a greedy "move the shorter wall" rule. Key concept: moving the taller wall can never increase area since width shrinks and height is capped by the shorter wall. Matters as proof that two-pointer greedy reasoning generalizes beyond sum problems. Prereq: #18. Approach: start at both ends, always move the pointer at the shorter height inward. Insight: this greedy step is provably safe — a classic exchange argument. Mistake: moving both pointers or the taller one. Variation: Trapping Rain Water. Companies: Amazon, Bloomberg.

#22. Trapping Rain Water — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Two pointers tracking running max height from both sides. Key concept: water above index i = min(maxLeft, maxRight) - height[i]. Matters as one of the most iconic hard problems, testable via two pointers, DP (precomputed max arrays), or monotonic stack — knowing all three shows depth. Prereq: #21. Approach: two pointers with leftMax/rightMax; advance the side with the smaller max, since that side's water level is determined. Insight: you never need the *global* max, only the smaller of the two current maxes. Mistake: trying to compute exact height contribution without realizing the min(leftMax,rightMax) trick. Variation: Trapping Rain Water II (2D, heap-based). Companies: Amazon, Google, Meta — extremely frequent.

#23. Sort Colors — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Dutch National Flag — three pointers (low, mid, high) partitioning into 3 groups in one pass. Matters as the canonical in-place partition-by-category problem. Prereq: #18. Approach: swap 0s to front, 2s to back, advance mid pointer conditionally. Insight: when swapping with high pointer, don't advance mid (need to recheck swapped-in value). Mistake: advancing mid after every swap regardless of direction. Variation: Sort an array of 0s, 1s, 2s with k categories. Companies: Meta, Microsoft.

#24. Remove Duplicates from Sorted Array — Easy · 🟢 · ⭐ ⭐ ⭐: Same-direction (slow/fast) pointers. Key concept: slow pointer marks the boundary of the deduplicated prefix. Matters as the intro to same-direction two-pointer (different from opposite-direction). Prereq: none. Approach: fast pointer scans, slow pointer only advances (and copies) on a new value. Insight: same template solves "remove element", "move zeroes". Mistake: confusing this with opposite-direction pattern. Variation: Remove Duplicates II (allow 2 copies), Move Zeroes. Companies: Microsoft, Amazon.

#25. Boats to Save People — Medium | 🟠 | ⭐ ⭐ ⭐: Greedy two pointers on sorted weights, pairing lightest with heaviest when they fit. Matters for combining two-pointer mechanics with a greedy proof. Prereq: #21. Approach: sort, two pointers from ends, if sum ⩽ limit move both, else only move heavy end. Insight: always try to pair the heaviest with someone — never wasteful to attempt. Mistake: not moving the heavy pointer every iteration. Companies: Google.

---

3. Sliding Window

Pattern Card

- **Recognition signals:** "contiguous subarray/substring", longest/shortest such that condition holds, "at most K distinct", "exactly K", repeating character constraints.
- **Core idea:** maintain a window `[left, right]`; expand right greedily, shrink left only when the window violates a constraint, tracking a running aggregate (count map, sum) incrementally instead of recomputing.
- **Typical complexity:** O(n) — each pointer moves forward at most n times.
- **Common traps:** recomputing window state from scratch each iteration (defeats the purpose); "exactly K" needs `atMost(K) - atMost(K-1)` trick; forgetting to shrink window fully before comparing best length.

#	Problem	Difficulty	Level	Priority
26	Best Time to Buy and Sell Stock	Easy	🟢	⭐⭐⭐⭐⭐
27	Longest Substring Without Repeating Characters	Medium	🔵	⭐⭐⭐⭐⭐
28	Longest Repeating Character Replacement	Medium	🟠	⭐⭐⭐⭐⭐
29	Permutation in String	Medium	🔵	⭐⭐⭐⭐
30	Minimum Window Substring	Hard	🔴	⭐⭐⭐⭐⭐
31	Sliding Window Maximum	Hard	🔴	⭐⭐⭐⭐⭐
32	Fruit Into Baskets	Medium	🔵	⭐⭐⭐
33	Subarrays with K Different Integers	Hard	🔴	⭐⭐⭐
34	Max Consecutive Ones III	Medium	🟠	⭐⭐⭐⭐

Problem Details

#26. Best Time to Buy and Sell Stock — Easy · 🟢 · ⭐⭐⭐⭐⭐: Implicit sliding window / single pass min-tracking. Key concept: track minimum price seen so far, compute profit at each step. Matters as the easiest gateway into "single pass with running state" thinking. Prereq: none. Approach: track `minPrice`, update `maxProfit = max(maxProfit, price-minPrice)`. Insight: this is really a size-varying window where left pointer = index of minimum. Mistake: computing all pairs $O(n^2)$. Variation: Buy/Sell Stock II (multiple transactions, greedy), III/IV (DP with k transactions), with cooldown. Companies: Amazon, Meta — universally asked.

#27. Longest Substring Without Repeating Characters — Medium · 🔵 · ⭐⭐⭐⭐⭐: Variable window + frequency/last-seen map. Key concept: shrink window whenever a repeat is found by jumping left pointer past the previous occurrence. Matters as the definitive variable-window template. Prereq: none. Approach: hash map of char→last index; when a repeat is found within window, move left to `lastIndex+1`. Insight: left pointer only ever moves forward — never reset to 0. Mistake: using a naive shrink loop instead of the O(1) jump (still correct but less optimal to explain). Variation: Longest Substring with At Most Two Distinct Characters. Companies: Amazon, Meta, Microsoft, Bloomberg — top-tier frequency.

#28. Longest Repeating Character Replacement — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Variable window with a "budget" constraint (k replacements). Key concept: window is valid while `windowLength - count(mostFrequentChar) <= k`. Matters for teaching "window validity via a derived quantity" rather than a direct count. Prereq: #27. Approach: track frequency map + max frequency char count; shrink only when constraint violated. Insight: `maxFreq` never needs to decrease when shrinking — an approximate/stale maxFreq still yields the correct final answer length. Mistake: trying to accurately recompute maxFreq on shrink (unnecessary complexity). Companies: Google, Meta.

#29. Permutation in String — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Fixed-size window + frequency map comparison. Key concept: slide a window the size of s1 across s2, compare frequency counts. Matters as the template for "does a fixed-size window match a target multiset" problems. Prereq: #27. Approach: frequency array of 26 letters for s1, sliding count for current window, compare via a match counter instead of full array comparison each step. Insight: maintain a `matches` counter that increments/decrements as characters enter/leave, avoiding O(26) comparisons per step. Mistake: comparing full 26-length arrays every iteration (still O(n·26), acceptable but less elegant). Variation: Find All Anagrams in a String. Companies: Microsoft.

#30. Minimum Window Substring — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Variable window with "cover all required characters" constraint. Key concept: expand until window covers target frequency map, then shrink from left while still valid, tracking the minimum. Matters as the hardest and most instructive sliding-window problem — combines expand/shrink with a "satisfied count" trick. Prereq: #27, #29. Approach: need-map of target chars, `have`/`need` counters tracking how many distinct chars currently satisfy their required count. Insight: increment `have` only when a char's window count *exactly* reaches its required count (not every time it's seen). Mistake: incrementing `have` on every occurrence, breaking the count logic. Companies: Meta, Uber, LinkedIn.

#31. Sliding Window Maximum — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Monotonic deque. Key concept: maintain a deque of indices in decreasing value order; front is always the current window max. Matters as the bridge between Sliding Window and Monotonic Queue topics. Prereq: #27. Approach: pop smaller values off the back before pushing; pop front if it falls outside window. Insight: each index is pushed/popped at most once → O(n) despite nested-looking logic. Mistake: using a max-heap (O(n log n), works but suboptimal) instead of a deque. Companies: Amazon, Google — classic hard.

#32. Fruit Into Baskets — Medium · 🔵 · ⭐ ⭐ ⭐: "At most 2 distinct types" variable window — a renamed version of the two-distinct-characters problem. Matters for pattern recognition across disguised problem statements. Prereq: #27. Approach: frequency map capped at 2 distinct keys, shrink when a 3rd type appears. Insight: recognizing that "baskets holding 2 fruit types" == "longest substring with at most 2 distinct chars" is the real skill being tested. Mistake: overcomplicating with two explicit basket variables instead of a general map. Companies: Google.

#33. Subarrays with K Different Integers — Hard · 🔴 · ⭐ ⭐ ⭐: "Exactly K" via `atMost(K) - atMost(K-1)`. Key concept: counting subarrays with *exactly* K distinct is hard directly, but "at most K" is a standard shrinking window, and exactly = atMost(K) - atMost(K-1). Matters as the canonical exactly-K trick, reusable in many counting-subarray problems. Prereq: #27, #32. Approach: implement `atMostKDistinct(k)` once, call twice. Insight: this subtraction trick converts many

"exactly" problems into two "at most" computations. Mistake: trying to directly track "exactly K" with ad hoc shrink conditions. Companies: Google, Meta.

#34. Max Consecutive Ones III — Medium · 🔴 · ⭐⭐⭐⭐: Variable window with a flip budget (identical shape to #28 but with 0/1 arrays). Matters for reinforcing the budgeted-window template with a simpler data type before revisiting #28/#30. Prereq: #27. Approach: track count of zeros in window; shrink when zero-count exceeds k. Insight: same "budget" abstraction as Longest Repeating Character Replacement. Mistake: not recognizing the shared template and re-deriving from scratch. Companies: Google, Facebook/Meta.

---

4. Binary Search

Pattern Card

- **Recognition signals:** sorted (or rotated-sorted) array, "find target/insertion point", "minimize/maximize the answer subject to a feasibility check", matrix with sorted rows/cols.
- **Core idea:** repeatedly halve a monotonic search space. Two flavors: (1) search for an exact value/boundary in sorted data, (2) "binary search on the answer" — guess a value, use a feasibility function to decide which half to keep.
- **Typical complexity:** O(log n) per search; O(n log n) if binary search is applied per element.
- **Common traps:** infinite loops from wrong `mid` rounding or wrong boundary update (`lo=mid` vs `lo=mid+1`); not recognizing a problem as "binary search on answer" because the array itself isn't sorted.

#	Problem	Difficulty	Level	Priority
35	Binary Search	Easy	🟢	⭐⭐⭐⭐⭐
36	Search Insert Position	Easy	🟢	⭐⭐⭐
37	Find First and Last Position of Element in Sorted Array	Medium	🔵	⭐⭐⭐⭐⭐
38	Search in Rotated Sorted Array	Medium	🔴	⭐⭐⭐⭐⭐
39	Find Minimum in Rotated Sorted Array	Medium	🔴	⭐⭐⭐⭐
40	Search a 2D Matrix	Medium	🔵	⭐⭐⭐⭐
41	Koko Eating Bananas	Medium	🔴	⭐⭐⭐⭐⭐
42	Capacity To Ship Packages Within D Days	Medium	🔴	⭐⭐⭐⭐
43	Split Array Largest Sum	Hard	🔴	⭐⭐⭐⭐
44	Median of Two Sorted Arrays	Hard	🔴	⭐⭐⭐⭐⭐
45	Find Peak Element	Medium	🔴	⭐⭐⭐

Problem Details

#35. Binary Search — Easy · 🟢 · ⭐⭐⭐⭐⭐: Standard template. Key concept: `lo<=hi`, `mid=lo+(hi-lo)/2`, adjust bounds. Matters as the base template everything else modifies. Prereq: none. Approach: classic. Insight: `lo+(hi-lo)/2` avoids overflow in languages with fixed-width ints. Mistake: `hi = nums.length` vs `nums.length-1` confusion. Companies: everywhere.

#36. Search Insert Position — Easy · 🟢 · ⭐⭐⭐: Lower-bound search. Key concept: find first index where `nums[i] >= target`. Matters as intro to boundary search (vs exact match). Prereq: #35. Approach: `lo<hi`, shrink to `[lo, mid]` or `[mid+1, hi]`. Insight: this "lo<hi, return lo" template is the universal lower-bound template. Mistake: conflating with exact-match template. Variation: bisect_left/right equivalents. Companies: Google.

#37. Find First and Last Position — Medium · 🔵 · ⭐⭐⭐⭐⭐: Two binary searches — lower bound and upper bound. Key concept: reuse the boundary-search template twice with different comparators. Matters as the canonical "binary search for a range" problem. Prereq: #36. Approach: search for leftmost occurrence, then rightmost occurrence. Insight: rightmost = (leftmost of target+1) - 1. Mistake: writing one messy combined search instead of two clean boundary searches. Companies: Meta, LinkedIn.

#38. Search in Rotated Sorted Array — Medium · 🟠 · ⭐⭐⭐⭐⭐: Modified binary search — determine which half is sorted, then decide if target lies in it. Matters as the most iconic "binary search on modified structure" problem. Prereq: #35. Approach: compare `nums[mid]` with `nums[lo]` to know which half is sorted, then range-check target against that half. Insight: at least one half around `mid` is always properly sorted — that's the invariant to exploit. Mistake: forgetting `<=` vs `<` when checking which side is sorted. Variation: with duplicates (Search in Rotated Sorted Array II), Find Minimum in Rotated Sorted Array. Companies: Amazon, Microsoft, Meta — very frequent.

#39. Find Minimum in Rotated Sorted Array — Medium · 🟠 · ⭐⭐⭐⭐: Binary search for rotation point. Key concept: compare `nums[mid]` to `nums[hi]` to decide which half contains the minimum. Matters for isolating the "find the pivot" sub-skill used inside #38. Prereq: #38. Approach: `if nums[mid] > nums[hi]: lo=mid+1 else hi=mid`. Insight: comparing against `hi` (not `lo`) simplifies the logic to two cases instead of four. Mistake: comparing against `lo` and mishandling edge cases. Companies: Amazon.

#40. Search a 2D Matrix — Medium · 🔵 · ⭐⭐⭐⭐: Treat a row-and-column sorted matrix as a flattened sorted array. Key concept: convert a 1D binary-search index into (row, col) via div/mod. Matters as the bridge from 1D to matrix binary search. Prereq: #35. Approach: binary search over `[0, rows*cols)`, map `mid` to `(mid/cols, mid%cols)`. Insight: only works when the matrix is fully sorted (last of row < first of next row); otherwise use staircase search. Mistake: applying this to a matrix that's only row/col-sorted independently (needs staircase search instead — see #40b variant). Companies: Amazon, Meta.

#41. Koko Eating Bananas — Medium · 🟠 · ⭐⭐⭐⭐⭐: Binary search on the answer with a feasibility check. Key concept: guess an eating speed, compute hours needed, binary search the minimum feasible speed. Matters as the definitive intro to "binary search on answer" — the single highest-leverage pattern in this section. Prereq: #35. Approach: search speed in `[1, max(piles)]`; feasibility = `sum(ceil(pile/speed)) <= h`. Insight: recognizing monotonicity ("if speed S works, any speed > S also works") is what licenses binary search here. Mistake: not identifying the

monotonic feasibility function, trying brute force instead. Variation: Capacity to Ship Packages, Split Array Largest Sum, Minimum Days to Make Bouquets. Companies: Google, Amazon — flagship "search space" problem.

#42. Capacity To Ship Packages Within D Days — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Binary search on answer, identical shape to #41. Matters for repetition/reinforcement — recognizing the same pattern in a differently-worded problem. Prereq: #41. Approach: search capacity in `[max(weights), sum(weights)]`; feasibility = can ship within D days at that capacity. Insight: the search bounds themselves encode the trivial min/max feasibility endpoints. Mistake: wrong bound initialization. Companies: Google, Facebook/Meta.

#43. Split Array Largest Sum — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Binary search on answer (harder feasibility function: greedy partition count). Matters as the hard-tier version proving mastery of the pattern; also solvable via DP, so it's a good "compare two approaches" problem. Prereq: #41, #42. Approach: search the max-subarray-sum value; feasibility = greedily count how many subarrays needed to keep each ⩽ candidate. Insight: same binary-search-on-answer skeleton as #41/#42 — only the feasibility function changes. Mistake: attempting a DP-only solution during time pressure when binary search is faster to code. Companies: Google, Meta.

#44. Median of Two Sorted Arrays — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Binary search for a partition point across two arrays. Key concept: binary search on the smaller array to find a partition such that left halves ⩽ right halves across both arrays. Matters as one of the most notorious hard problems — a strong signal of deep binary-search fluency. Prereq: #35, #37. Approach: binary search partition index in the smaller array; derive the other array's partition from total length; compare boundary elements. Insight: always binary search on the *shorter* array to keep the partition bound valid. Mistake: mishandling even/odd total length or empty-partition edge values (use ±infinity sentinels). Companies: Google, Apple, Microsoft — famously hard.

#45. Find Peak Element — Medium · 🔴 · ⭐ ⭐ ⭐: Binary search using local slope instead of full sortedness. Key concept: move toward the side with the larger neighbor — a peak is guaranteed to exist that direction. Matters for showing binary search applies even without a fully sorted array, only a "directional" invariant. Prereq: #35. Approach: `if nums[mid] < nums[mid+1]: lo=mid+1 else hi=mid`. Insight: the "any local increase guarantees a peak ahead" argument is the generalizable insight. Mistake: assuming the whole array must be monotonic. Companies: Google, Amazon.

---

5. Linked Lists

Pattern Card

- **Recognition signals:** "reverse a list", "detect a cycle", "find the middle", "merge sorted lists", "remove Nth node", problems mentioning `ListNode`.
- **Core idea:** pointer manipulation with careful handling of `next` references; fast/slow pointers solve middle-finding and cycle-detection in one pass; dummy head nodes simplify edge cases (empty list, removing head).
- **Typical complexity:** O(n) time, O(1) space for most; recursion trades O(1) space for O(n) call stack.
- **Common traps:** losing a reference to `next` before reassigning it during reversal; forgetting a dummy head causes special-casing the head node; not advancing pointers correctly in

fast/slow (fast moves 2 steps, slow 1).

#	Problem	Difficulty	Level	Priority
46	Reverse Linked List	Easy	🟢	⭐⭐⭐⭐⭐
47	Merge Two Sorted Lists	Easy	🟢	⭐⭐⭐⭐⭐
48	Linked List Cycle	Easy	🟢	⭐⭐⭐⭐⭐
49	Linked List Cycle II	Medium	🔵	⭐⭐⭐⭐
50	Remove Nth Node From End of List	Medium	🔵	⭐⭐⭐⭐⭐
51	Reorder List	Medium	🟠	⭐⭐⭐⭐
52	Copy List with Random Pointer	Medium	🟠	⭐⭐⭐⭐
53	Add Two Numbers	Medium	🔵	⭐⭐⭐⭐⭐
54	LRU Cache	Medium	🔴	⭐⭐⭐⭐⭐
55	Merge k Sorted Lists	Hard	🔴	⭐⭐⭐⭐⭐
56	Reverse Nodes in k-Group	Hard	🔴	⭐⭐⭐⭐
57	Intersection of Two Linked Lists	Easy	🟢	⭐⭐⭐

Problem Details

#46. Reverse Linked List — Easy · 🟢 · ⭐⭐⭐⭐⭐: Iterative pointer reversal. Key concept: track `prev`, `curr`, `next` and flip the arrow each step. Matters as the foundational operation reused inside dozens of harder list problems. Prereq: none. Approach: iterate, save `next`, set `curr.next=prev`, shift all three forward. Insight: also learn the recursive version — `reverse(head.next)` then fix `head.next.next=head`. Mistake: losing the `next` reference before reassigning. Variation: reverse between positions m and n, reverse in k-groups. Companies: everywhere — most-asked list question.

#47. Merge Two Sorted Lists — Easy · 🟢 · ⭐⭐⭐⭐⭐: Dummy-head merge, same idea as merging sorted arrays. Key concept: use a dummy node to avoid special-casing the new head. Matters as the base case for Merge k Sorted Lists. Prereq: none. Approach: compare heads, attach smaller, advance; attach remainder at the end. Insight: dummy-head trick generalizes to almost all list-building problems. Mistake: forgetting to attach the leftover tail of the non-empty list. Companies: Amazon, Microsoft.

#48. Linked List Cycle — Easy · 🟢 · ⭐⭐⭐⭐⭐: Floyd's fast/slow pointer. Key concept: if a cycle exists, fast pointer eventually laps slow pointer. Matters as the entry point to Floyd's algorithm, reused in #49 and array-based cycle problems (Find the Duplicate Number). Prereq: none. Approach: `slow` moves 1, `fast` moves 2; if they meet, cycle exists. Insight: no cycle ⇒ fast

reaches null first — that's the loop termination condition. Mistake: not checking `fast` and `fast.next` both non-null before advancing. Companies: Amazon, Bloomberg.

#49. Linked List Cycle II — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Floyd's algorithm extended to find the cycle's start node. Key concept: after slow/fast meet, reset one pointer to head; both now move 1 step at a time and meet exactly at the cycle start (provable via distance algebra). Matters as a deeper mathematical extension of #48. Prereq: #48. Approach: detect meeting point, then two single-step pointers from head and meeting point. Insight: the distance from head to cycle start equals the distance from the meeting point to the cycle start (going forward). Mistake: not knowing this proof, unable to justify the "reset to head" step in an interview. Companies: Amazon.

#50. Remove Nth Node From End of List — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Two pointers with a fixed gap. Key concept: advance a lead pointer n steps first, then move both together — lead hits the end exactly when trail is at the removal point. Matters as the template for "Nth from end" in one pass. Prereq: #46. Approach: dummy head, lead pointer n+1 steps ahead of trail. Insight: using a dummy head cleanly handles removing the actual head node (n == length). Mistake: off-by-one in the initial gap. Companies: Meta, Amazon.

#51. Reorder List — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Combine three sub-skills: find middle (fast/slow), reverse second half, merge alternately. Matters as a great "compose multiple list techniques" interview problem. Prereq: #46, #48. Approach: split at middle, reverse second half, weave the two halves together. Insight: recognizing decomposition into known sub-problems is the actual skill tested. Mistake: mishandling the middle split for odd vs even length lists. Companies: Meta, Amazon.

#52. Copy List with Random Pointer — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Hash map (old→new) or O(1)-space interleaving trick. Key concept: build new nodes while preserving `random` pointer references via a lookup structure (or by interleaving copies into the original list). Matters for combining linked lists with hashing, and for the elegant O(1)-space interleaving alternative. Prereq: #47. Approach (map): one pass to create nodes + map, second pass to wire `next`/`random`. Approach (O(1)): interleave copy nodes into original list, wire randoms using `node.next`, then detach. Insight: the interleaving trick avoids extra space by temporarily doubling the list. Mistake: wiring `random` before all nodes exist. Companies: Meta, Amazon, Bloomberg.

#53. Add Two Numbers — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Digit-by-digit addition with carry, represented as reversed linked lists. Key concept: simultaneous traversal with carry propagation, similar to elementary-school addition. Matters as the classic "simulate arithmetic on a list" problem. Prereq: #47. Approach: dummy head, sum digits + carry at each step, create new node for `sum % 10`, carry `sum / 10`. Insight: continue looping while either list has nodes OR carry is nonzero. Mistake: forgetting a trailing carry after both lists end. Variation: Add Two Numbers II (non-reversed, use a stack). Companies: Amazon, Microsoft — very common.

#54. LRU Cache — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Doubly linked list + hash map design. Key concept: hash map gives O(1) lookup, doubly linked list gives O(1) reordering/eviction — together they support O(1) `get`/`put`. Matters as one of the most-asked design problems across all of MAANG, testing whether you can combine data structures for combined guarantees. Prereq: #46, #47 conceptually. Approach: map key→node; on access, unlink node and move to front (most-recently-used end); evict from the tail on overflow. Insight: use dummy head AND tail sentinels to

avoid null-checks when unlinking. Mistake: forgetting to update the map on eviction, or mishandling capacity=0. Variation: LFU Cache (harder — frequency buckets of doubly linked lists). Companies: Amazon, Google, Meta, Uber — a top design question.

#55. Merge k Sorted Lists — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Min-heap over list heads, or divide-and-conquer pairwise merge. Key concept: repeatedly extract the globally smallest head from k lists. Matters as the natural generalization of #47, and a bridge into the Heap topic. Prereq: #47. Approach (heap): push all k heads into a min-heap keyed by value, pop-and-push-next repeatedly. Approach (D&C): merge lists pairwise like merge sort, `O(n log k)`. Insight: divide-and-conquer avoids heap overhead and is often the more impressive answer. Mistake: naively concatenating and sorting (`O(n log n)` instead of `O(n log k)`, weaker answer). Companies: Google, Amazon, Meta — classic hard.

#56. Reverse Nodes in k-Group — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Reverse a linked list in fixed-size chunks. Key concept: reuse the #46 reversal routine on each k-sized segment, recursively or iteratively linking segments together. Matters as a hard problem that's really "compose reversal + segmenting," a good test of careful pointer bookkeeping. Prereq: #46. Approach: check k nodes exist ahead; reverse that segment; recursively process the remainder and attach. Insight: always count-check that a full group of k exists before reversing (leave remainder untouched). Mistake: reversing a partial final group when it shouldn't be reversed. Companies: Meta, Amazon.

#57. Intersection of Two Linked Lists — Easy · 🟢 · ⭐ ⭐ ⭐: Two-pointer length-equalization trick. Key concept: walk both pointers, and when one reaches the end, redirect it to the other list's head — they meet at the intersection after at most `lenA+lenB` steps. Matters as an elegant O(1)-space alternative to hashing all nodes. Prereq: none. Approach: pointer A traverses A then B; pointer B traverses B then A; they align and meet. Insight: this "switch lists" trick effectively equalizes the remaining distance to the intersection for both pointers. Mistake: comparing node values instead of node references (must be reference equality). Companies: Amazon, Microsoft.

---

6. Stack & Monotonic Stack

Pattern Card

- **Recognition signals:** "valid parentheses/expression", "next greater/smaller element", histogram/skyline problems, "daily temperatures", undo/backtrack-style processing, expression evaluation.
- **Core idea:** a stack naturally models nested/LIFO structure (matching brackets, recursive expressions). A *monotonic* stack (kept increasing or decreasing) lets you find, for every element, the nearest greater/smaller neighbor in O(n) total instead of $O(n^2)$.
- **Typical complexity:** O(n) — each element pushed/popped at most once.
- **Common traps:** popping in the wrong direction (increasing vs decreasing stack) for "next greater" vs "next smaller"; forgetting to store indices (not just values) when the answer needs distance; not handling leftover stack elements after the main loop.

#	Problem	Difficulty	Level	Priority
58	Valid Parentheses	Easy	🟢	⭐⭐⭐⭐⭐
59	Min Stack	Medium	🔵	⭐⭐⭐⭐⭐
60	Evaluate Reverse Polish Notation	Medium	🔵	⭐⭐⭐⭐
61	Daily Temperatures	Medium	🔵	⭐⭐⭐⭐⭐
62	Next Greater Element I	Easy	🟢	⭐⭐⭐
63	Car Fleet	Medium	🟠	⭐⭐⭐
64	Largest Rectangle in Histogram	Hard	🔴	⭐⭐⭐⭐⭐
65	Maximal Rectangle	Hard	🔴	⭐⭐⭐
66	Basic Calculator	Hard	🔴	⭐⭐⭐⭐
67	Decode String	Medium	🟠	⭐⭐⭐⭐
68	Remove K Digits	Medium	🟠	⭐⭐⭐

Problem Details

#58. Valid Parentheses — Easy · 🟢 · ⭐⭐⭐⭐⭐: Stack-based bracket matching. Key concept: push opens, pop-and-compare on closes. Matters as the canonical stack warm-up. Prereq: none. Approach: map closing→opening, push opens, pop on close and check match. Insight: an empty stack at the end (and never popping empty mid-way) is the full validity condition. Mistake: forgetting to check the stack is empty at the very end. Companies: everywhere.

#59. Min Stack — Medium · 🔵 · ⭐⭐⭐⭐⭐: Auxiliary stack tracking running minimum. Key concept: push `(value, currentMin)` pairs, or maintain a parallel min-stack. Matters as a design problem testing "augment a stack with extra O(1) query capability." Prereq: #58. Approach: parallel stack pushes `min(newVal, minStack.top())` each time. Insight: no need to pop conditionally from the min-stack — pushing a duplicate min on every push/pop keeps both stacks the same length and in sync. Mistake: trying to recompute the min by scanning (defeats O(1)). Companies: Amazon, Meta — frequent design question.

#60. Evaluate Reverse Polish Notation — Medium · 🔵 · ⭐⭐⭐⭐: Stack-based expression evaluation. Key concept: push operands, pop two and apply operator on encountering one. Matters as the intro to expression-parsing with stacks, leading into #66. Prereq: #58. Approach: iterate tokens, push numbers, pop two + operate + push result on operators. Insight: operand order matters for `-`/`/` — pop second-then-first correctly. Mistake: swapping operand order for non-commutative ops. Companies: LinkedIn.

#61. Daily Temperatures — Medium · 🔵 · ⭐⭐⭐⭐⭐: Monotonic decreasing stack of indices. Key concept: maintain a stack of indices whose temperatures are decreasing; pop and resolve whenever a warmer day appears. Matters as the cleanest possible "next greater element, with

distance" template. Prereq: #58. Approach: iterate, while stack top's temp < current, pop and set `answer[popped] = i - popped`. Insight: storing indices (not values) is essential since the answer needs distance, not the value itself. Mistake: storing values, then being unable to compute distance. Variation: Next Greater Element I/II, Online Stock Span. Companies: Amazon, Google — extremely popular monotonic stack intro.

#62. Next Greater Element I — Easy · 🟢 · ⭐ ⭐ ⭐: Monotonic stack + hash map lookup. Key concept: precompute "next greater" for every element of `nums2` via monotonic stack, then answer queries for `nums1` via a map. Matters as a simplified version of #61 for building initial monotonic-stack confidence. Prereq: #61. Approach: decreasing stack over nums2, map each resolved element to its next-greater value. Insight: separate the "compute for all" step from the "answer queries" step. Mistake: trying to solve per-query instead of precomputing once. Companies: Amazon.

#63. Car Fleet — Medium · 🔴 · ⭐ ⭐ ⭐: Monotonic stack sorted by position, using arrival-time comparisons to merge into fleets. Key concept: process cars from closest-to-target backward; a car merges into the fleet ahead if its time-to-target is ⩽ that fleet's time. Matters as a disguised monotonic-stack problem (not obviously stack-shaped from the prompt). Prereq: #61. Approach: sort by position descending, maintain a stack of fleet arrival times, push only when the current car's time exceeds the stack top (i.e., it forms a new fleet). Insight: sorting order + "cannot pass" physical constraint is what creates the monotonic structure. Mistake: sorting in the wrong direction. Companies: Google.

#64. Largest Rectangle in Histogram — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Monotonic increasing stack of indices, resolving width spans. Key concept: for each bar, find the largest rectangle where that bar is the shortest — determined by nearest smaller bar to the left and right. Matters as one of the most important hard-tier monotonic stack problems, directly setting up Maximal Rectangle. Prereq: #61. Approach: push indices while heights increasing; when a smaller bar appears, pop and compute area using the popped height and the width between the new stack top and current index. Insight: appending a sentinel `0`-height bar at the end forces all remaining stack entries to resolve. Mistake: mis-computing width when the stack becomes empty after a pop (means it extends to index -1). Companies: Google, Amazon, Meta — a flagship hard problem.

#65. Maximal Rectangle — Hard · 🔴 · ⭐ ⭐ ⭐: Row-by-row histogram reduction + #64. Key concept: treat each row of a binary matrix as the "heights" of a histogram (stacking consecutive 1s vertically) and apply Largest Rectangle in Histogram per row. Matters for showing 2D problems can decompose into repeated 1D subproblems. Prereq: #64. Approach: build a running height array per row, run #64's algorithm on each row. Insight: recognizing the reduction to #64 is the entire difficulty — the code itself is a direct reuse. Mistake: not realizing histogram heights reset to 0 whenever a `0` appears in the matrix. Companies: Google.

#66. Basic Calculator — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Stack-based expression parsing with parentheses and signs. Key concept: use a stack to save `(result, sign)` context when entering a parenthesis, restore on exit. Matters as the deepest expression-parsing stack problem, often extended to full calculators with `*`/`/`. Prereq: #58, #60. Approach: maintain running result and sign; push onto stack when `(` encountered, pop and combine when `)` encountered. Insight: handling multi-digit numbers and consecutive signs correctly is the main implementation challenge, not the stack logic itself. Mistake: mishandling operator precedence when `*`/`/` are added (Basic Calculator II/III). Companies: Google, Meta.

#67. Decode String — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Stack-based nested-structure decoding. Key concept: push `(currentString, repeatCount)` onto a stack when entering a bracket, pop and repeat/concatenate when closing. Matters as the template for any "nested repeated structure" problem. Prereq: #58. Approach: build number and string buffers; on `[` push state and reset buffers; on `]` pop and repeat. Insight: recognize this as a generalized bracket-matching problem where the "payload" carried on the stack is state, not just a symbol. Mistake: forgetting multi-digit repeat counts. Companies: Amazon, Google.

#68. Remove K Digits — Medium · 🔴 · ⭐ ⭐ ⭐: Monotonic increasing stack, greedily removing larger preceding digits. Key concept: to make the smallest number after removing k digits, pop off any digit that's larger than the current one (greedy + stack), as long as removals remain. Matters as a greedy-plus-monotonic-stack combination. Prereq: #61. Approach: iterate digits, pop stack while top > current digit and k>0; push current; trim remaining k from the end if needed. Insight: leading zeros and leftover-k-after-loop are the two edge cases to handle explicitly. Mistake: forgetting to remove from the end if k still > 0 after the scan. Companies: Google, Amazon.

---

7. Queue & Monotonic Queue

Pattern Card

- **Recognition signals:** level-order traversal (BFS), "sliding window maximum/minimum", task scheduling with cooldowns, anything processed in FIFO arrival order.
- **Core idea:** a queue models FIFO/level-by-level processing; a deque used as a "monotonic queue" maintains window extrema in O(1) amortized per operation by discarding elements that can never become the answer.
- **Typical complexity:** O(n) — each element enters/leaves the queue once.
- **Common traps:** using a plain queue where a deque (double-ended) is required; forgetting to evict out-of-window indices at the front; confusing monotonic queue with monotonic stack (queue evicts from both ends, stack only from the top).

#	Problem	Difficulty	Level	Priority
69	Number of Recent Calls	Easy	🟢	⭐ ⭐
70	Design Circular Queue	Medium	🔵	⭐ ⭐
71	Sliding Window Maximum	Hard	🔴	⭐ ⭐ ⭐ ⭐ ⭐
72	Task Scheduler	Medium	🔴	⭐ ⭐ ⭐ ⭐ ⭐
73	Binary Tree Level Order Traversal	Medium	🔵	⭐ ⭐ ⭐ ⭐ ⭐
74	Rotting Oranges	Medium	🔴	⭐ ⭐ ⭐ ⭐ ⭐
75	Design Hit Counter	Medium	🔵	⭐ ⭐

Problem Details

#69. Number of Recent Calls — Easy · 🟢 · ⭐ ⭐: Plain queue with eviction of stale timestamps. Key concept: a request queue where you pop everything older than a time window. Matters as the simplest FIFO-with-eviction warm-up. Prereq: none. Approach: push new timestamp, pop from front while `front < t-3000`. Insight: this is a fixed-time (not fixed-count) sliding window over a queue. Mistake: using a list with O(n) removal instead of a true queue/deque. Companies: Google.

#70. Design Circular Queue — Medium · 🔵 · ⭐ ⭐: Array-based circular buffer design. Key concept: use modulo arithmetic on head/tail pointers to wrap around a fixed-size array. Matters as a fundamental data-structure design exercise. Prereq: none. Approach: track `head`, `count`, and capacity; enqueue/dequeue update head/tail via modulo. Insight: tracking a `count` avoids the classic full-vs-empty ambiguity when head==tail. Mistake: not handling the full/empty ambiguity at all. Companies: Google.

#71. Sliding Window Maximum — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: (see also #31) Monotonic deque holding indices in decreasing value order. Matters enough to appear in both Sliding Window and Queue topics — it is the definitive monotonic-queue problem. Prereq: understand deques. Approach: as in #31. Insight: the deque simultaneously handles "evict out-of-window" (front) and "maintain decreasing order" (back). Mistake: using two separate structures instead of one deque. Companies: Amazon, Google.

#72. Task Scheduler — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐ ⭐: Greedy + queue simulation, or a closed-form frequency formula. Key concept: schedule the most frequent tasks first, using idle slots (or a cooldown queue) to respect the cooldown constraint. Matters as a strong greedy/queue hybrid frequently asked at MAANG. Prereq: heap basics helpful. Approach (formula): `max(len(tasks), (maxFreq-1)*(n+1) + countOfMaxFreqTasks)`. Approach (simulation): max-heap of counts + a cooldown queue holding `(task, availableTime)`. Insight: the formula only works because you can prove the most frequent task determines the schedule's lower bound; the simulation approach is safer to derive under pressure. Mistake: applying the formula without understanding why it can undercount when many tasks tie for max frequency. Companies: Amazon, Meta — very common.

#73. Binary Tree Level Order Traversal — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: BFS using a queue, processing one full level at a time. Key concept: snapshot `len(queue)` at the start of each level to know how many nodes belong to it. Matters as the canonical BFS template for trees, reused everywhere. Prereq: none. Approach: queue starts with root; loop while queue non-empty, process `levelSize` nodes, enqueue children. Insight: capturing level size *before* enqueuing children is what separates levels cleanly. Mistake: not snapshotting level size, causing levels to blur together. Variation: zigzag level order, right side view, average of levels. Companies: Amazon, Meta, Microsoft — one of the most common tree questions.

#74. Rotting Oranges — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐ ⭐: Multi-source BFS on a grid. Key concept: start BFS simultaneously from *all* initially rotten oranges, tracking the number of minutes/levels until no fresh oranges remain reachable. Matters as the template for multi-source BFS, a very common grid pattern. Prereq: #73. Approach: enqueue all rotten cells first, BFS level by level, decrement a fresh-orange counter; return level count if 0 fresh remain, else -1. Insight: starting BFS from multiple sources at once (rather than looping single-source BFS) is what makes this efficient and correct. Mistake: running BFS separately per rotten orange (wrong "closest rotten" semantics and slower). Companies: Amazon, Google.

#75. Design Hit Counter — Medium · 🔵 · ⭐ ⭐: Queue-based fixed-time-window counting, same idea as #69 generalized to a design class. Matters for practicing queue-based design under a time-window constraint. Prereq: #69. Approach: queue of timestamps, evict everything older than 300s on each `getHits` call. Insight: for high-QPS variants, bucket counts by second in a circular array instead of a raw queue (O(1) instead of O(n) eviction). Mistake: not discussing the high-throughput follow-up. Companies: Google, Twitter.

---

8. Recursion & Backtracking

Pattern Card

- **Recognition signals:** "generate all subsets/permutations/combinations", "find all valid arrangements satisfying constraints", grid path exploration, N-Queens-style placement.
- **Core idea:** explore a decision tree by making a choice, recursing, then undoing the choice (backtrack) to try the next option. Pruning invalid branches early is what makes it tractable.
- **Typical complexity:** exponential in the worst case (O(2^n) subsets, O(n!) permutations) — the goal is correct pruning, not beating the theoretical bound.
- **Common traps:** forgetting to undo state (`pop()`/unmark visited) after recursing; not deduplicating when input has duplicates; passing mutable state by reference and appending it directly to the result (must deep-copy).

#	Problem	Difficulty	Level	Priority
76	Subsets	Medium	🔵	⭐ ⭐ ⭐ ⭐ ⭐
77	Subsets II	Medium	🟠	⭐ ⭐ ⭐ ⭐
78	Permutations	Medium	🔵	⭐ ⭐ ⭐ ⭐ ⭐
79	Permutations II	Medium	🟠	⭐ ⭐ ⭐
80	Combinations	Medium	🔵	⭐ ⭐ ⭐ ⭐
81	Combination Sum	Medium	🔵	⭐ ⭐ ⭐ ⭐ ⭐
82	Combination Sum II	Medium	🟠	⭐ ⭐ ⭐ ⭐
83	Word Search	Medium	🟠	⭐ ⭐ ⭐ ⭐ ⭐
84	Palindrome Partitioning	Medium	🟠	⭐ ⭐ ⭐ ⭐
85	N-Queens	Hard	🔴	⭐ ⭐ ⭐ ⭐
86	Sudoku Solver	Hard	🔴	⭐ ⭐ ⭐
87	Letter Combinations of a Phone Number	Medium	🔵	⭐ ⭐ ⭐ ⭐
88	Generate Parentheses	Medium	🔵	⭐ ⭐ ⭐ ⭐ ⭐

Problem Details

#76. Subsets — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: The base backtracking template — include/exclude each element. Key concept: at each index, branch into "take it" and "don't take it." Matters as the entry point to the entire backtracking topic. Prereq: recursion basics. Approach: recurse with a `start` index, add current path to result at every call, then loop `i=start..n` choosing/unchoosing each element. Insight: this "add-at-every-node" recursion tree directly generates the power set. Mistake: appending a reference to the mutable path list instead of a copy. Companies: Amazon, Meta, Google.

#77. Subsets II — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: Subsets with duplicate elements. Key concept: sort input, skip a duplicate value at the same recursion depth (`i>start and nums[i]==nums[i-1]: continue`). Matters as the standard duplicate-handling template reused across #79, #82, #84. Prereq: #76. Approach: sort first, apply the "skip same value at same level" rule. Insight: sorting is what makes duplicates adjacent and skippable. Mistake: skipping duplicates across different recursion depths (which is actually valid and shouldn't be skipped). Companies: Meta.

#78. Permutations — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Backtracking with a "used" tracker (or swap-based in-place generation). Key concept: at each level, try every unused element. Matters as the second foundational backtracking template (choose-from-remaining, vs. include/exclude in #76). Prereq: #76. Approach: maintain a boolean `used[]` array, or swap current index with each later index and recurse (`swap` avoids extra space). Insight: swap-based permutation avoids needing a separate `used` array. Mistake: forgetting to swap back after recursing (breaking backtracking invariant). Companies: Amazon, Google, Meta.

#79. Permutations II — Medium · 🟠 · ⭐ ⭐ ⭐: Permutations with duplicates. Key concept: sort + skip duplicate values at the same recursion level using a `used[]` array (the swap trick doesn't directly generalize). Matters for reinforcing dedup logic in the permutation context. Prereq: #78, #77. Approach: sort, use `used[]`, skip `nums[i]==nums[i-1] and not used[i-1]`. Insight: the `not used[i-1]` condition specifically prevents picking the second duplicate before the first at the same level. Mistake: getting the skip condition backwards (`used[i-1]` instead of `not used[i-1]`). Companies: Meta.

#80. Combinations — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Choose k elements from n without repetition or order. Key concept: same `start`-index recursion as Subsets, but only add to result when `path.length==k`. Matters as a size-constrained variant of the core template. Prereq: #76. Approach: recurse with start index, prune early if remaining elements can't fill `k`. Insight: pruning (`if n-start+1 < k-len(path): return`) meaningfully speeds up large inputs. Mistake: no pruning, causing unnecessary recursion. Companies: Google.

#81. Combination Sum — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Unbounded choice backtracking (elements reusable) with a sum target. Key concept: recurse with the same `start` index (not `start+1`) to allow reuse, subtracting from a remaining target. Matters as the template for "reusable elements" backtracking, contrasted with #82's "each used once." Prereq: #76. Approach: at each index, either take it again (recurse with same index) or move to next index. Insight: passing the *same* start index (not incrementing) is the single line that enables reuse. Mistake: incrementing start unconditionally, preventing reuse of the same number. Companies: Amazon, Meta, Snapchat.

#82. Combination Sum II — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: Each element used once, with duplicates in input, target sum. Key concept: sort + skip-duplicate-at-same-level (like #77) combined with

`start+1` (no reuse). Matters as the "no reuse + has duplicates" combination completing the Combination Sum family. Prereq: #81, #77. Approach: sort, standard skip-duplicate rule, recurse with `start+1`. Insight: this is a direct merge of the #77 dedup pattern and #81 skeleton. Mistake: mixing up when to increment start vs not (compare carefully against #81). Companies: Amazon.

#83. Word Search — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Grid DFS/backtracking with a visited marker. Key concept: DFS from each starting cell, marking visited cells temporarily and unmarking on backtrack. Matters as the definitive grid-backtracking problem, frequently seen at MAANG. Prereq: #76 for backtracking mental model. Approach: DFS trying all 4 directions, matching the next character; mark grid cell (e.g., overwrite with `#`) then restore it after recursing. Insight: mutating the grid in place as a visited marker avoids extra space for a separate visited matrix. Mistake: forgetting to restore/unmark the cell after the recursive call returns. Variation: Word Search II (multiple words via Trie + backtracking). Companies: Amazon, Microsoft, Meta.

#84. Palindrome Partitioning — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Backtracking over partition points + palindrome check. Key concept: try every prefix; if it's a palindrome, recurse on the remainder. Matters as a string-partitioning backtracking template, and a natural bridge into DP (Palindrome Partitioning II asks for minimum cuts). Prereq: #76. Approach: at each start index, loop end index, check `s[start:end]` is palindrome, recurse if so. Insight: precomputing a palindrome DP table (`isPal[i][j]`) turns repeated O(n) checks into O(1) lookups, a huge speedup. Mistake: recomputing palindrome checks with fresh O(n) scans each time instead of memoizing. Companies: Amazon, Meta.

#85. N-Queens — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Row-by-row backtracking with column/diagonal conflict tracking. Key concept: place one queen per row, track used columns and both diagonals (`row-col`, `row+col`) as sets for O(1) conflict checks. Matters as the flagship constraint-satisfaction backtracking problem. Prereq: #76, #83. Approach: recurse row by row, try each column, check column/diagonal sets before placing. Insight: diagonals are uniquely identified by `row-col` (one diagonal direction) and `row+col` (the other) — this is the key trick avoiding O(n) conflict scans. Mistake: checking conflicts by scanning the board instead of using O(1) set lookups. Companies: Google, Microsoft.

#86. Sudoku Solver — Hard · 🔴 · ⭐ ⭐ ⭐: Backtracking with row/col/box constraint sets (extends Valid Sudoku, #7). Key concept: try digits 1-9 in each empty cell, checking row/col/box sets, backtrack on dead ends. Matters as a comprehensive constraint-backtracking exercise combining #7's hashing with #85's placement logic. Prereq: #7, #85. Approach: precompute row/col/box used-digit sets, DFS over empty cells trying valid digits. Insight: picking the empty cell with the *fewest* valid candidates first (MRV heuristic) dramatically speeds up real-world solving, though a naive left-to-right scan also passes LeetCode. Mistake: not backtracking cleanly (removing the digit from tracking sets on failure). Companies: Uber, Apple.

#87. Letter Combinations of a Phone Number — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Backtracking over a fixed mapping per digit. Key concept: recurse index by index, branching over each digit's mapped letters. Matters as a clean, small backtracking example often used as a warm-up in interviews. Prereq: #76. Approach: digit→letters map, recurse building a path string. Insight: this is functionally identical to generating a Cartesian product of the digits' letter sets. Mistake: overcomplicating with iterative BFS-style queue building when simple recursion suffices. Companies: Amazon, Uber.

#88. Generate Parentheses — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Backtracking with a validity constraint (open count ⩾ close count at all times). Key concept: only add `(` if `open<n`, only add `)` if `close<open`. Matters as the cleanest example of "prune using a simple counting invariant" rather than checking full validity at the end. Prereq: #76. Approach: recurse tracking counts of open/close used so far, base case when `path.length==2n`. Insight: the two guard conditions alone guarantee every generated string is valid — no separate validation step needed. Mistake: generating all 2^(2n) strings and filtering valid ones afterward (correct but far less efficient / weaker interview answer). Companies: Amazon, Meta, Google, Bloomberg.

---

9. Trees

Pattern Card

- **Recognition signals:** anything with `TreeNode`, "binary search tree," "path sum," "lowest common ancestor," "serialize/deserialize," "diameter/height," tree construction from traversals.
- **Core idea:** most tree problems reduce to DFS (pre/in/post-order) or BFS (level order), often by defining a recursive function that returns information to its parent (height, whether a property holds, etc.) — "think about what one call should return, then trust the recursion."
- **Typical complexity:** O(n) time, O(h) recursion stack (h = height; O(log n) balanced, O(n) skewed).
- **Common traps:** confusing pre/in/post-order use cases; forgetting BST in-order traversal is sorted; not handling `null` base cases; mutating global state instead of returning values cleanly (works, but messier and more error-prone).

#	Problem	Difficulty	Level	Priority
89	Invert Binary Tree	Easy	🟢	⭐⭐⭐⭐⭐
90	Maximum Depth of Binary Tree	Easy	🟢	⭐⭐⭐⭐⭐
91	Same Tree	Easy	🟢	⭐⭐⭐
92	Subtree of Another Tree	Easy	🟢	⭐⭐⭐
93	Diameter of Binary Tree	Easy	🔵	⭐⭐⭐⭐⭐
94	Balanced Binary Tree	Easy	🔵	⭐⭐⭐⭐
95	Lowest Common Ancestor of a Binary Search Tree	Medium	🔵	⭐⭐⭐⭐
96	Lowest Common Ancestor of a Binary Tree	Medium	🟠	⭐⭐⭐⭐⭐
97	Binary Tree Right Side View	Medium	🔵	⭐⭐⭐⭐
98	Count Good Nodes in Binary Tree	Medium	🔵	⭐⭐⭐
99	Validate Binary Search Tree	Medium	🟠	⭐⭐⭐⭐⭐
100	Kth Smallest Element in a BST	Medium	🟠	⭐⭐⭐⭐
101	Construct Binary Tree from Preorder and Inorder Traversal	Medium	🟠	⭐⭐⭐⭐⭐
102	Binary Tree Maximum Path Sum	Hard	🔴	⭐⭐⭐⭐⭐
103	Serialize and Deserialize Binary Tree	Hard	🔴	⭐⭐⭐⭐⭐
104	Vertical Order Traversal of a Binary Tree	Hard	🔴	⭐⭐⭐

Problem Details

#89. Invert Binary Tree — Easy · 🟢 · ⭐⭐⭐⭐⭐: Simple recursive swap of left/right children. Matters historically as a famous "can you code at all" filter question and the simplest recursive-tree template. Prereq: none. Approach: swap `node.left`/`node.right`, recurse on both. Insight: post-order vs pre-order swap timing doesn't matter here — both work. Mistake: forgetting the null base case. Companies: Google (famously).

#90. Maximum Depth of Binary Tree — Easy · 🟢 · ⭐⭐⭐⭐⭐: `1 + max(depth(left), depth(right))`. Matters as the template for "return info from children, combine at parent" recursion used throughout the topic. Prereq: none. Approach: recursive, base case null returns 0. Insight: this exact shape (post-order aggregation) reappears in diameter, balanced-check, and max path sum. Mistake: off-by-one (counting edges vs nodes). Companies: everywhere.

#91. Same Tree — Easy · 🟢 · ⭐⭐⭐: Simultaneous recursive comparison of two trees. Matters as a warm-up for structural tree comparison, needed for #92. Prereq: none. Approach: both null → true; one null → false; compare values then recurse both children. Insight: the "compare structure +

value at every node" template extends directly to #92 and #99. Mistake: only comparing values without checking structural null-alignment. Companies: Bloomberg.

#92. Subtree of Another Tree — Easy · 🟢 · ⭐ ⭐ ⭐: For every node in the main tree, check if the subtree rooted there `isSameTree` (#91) as the target. Matters for showing composition of a previously solved subroutine. Prereq: #91. Approach: recurse over main tree; at each node call `isSameTree(node, subRoot)`. Insight: naive complexity is O(m·n) but is generally accepted; string-serialization + substring-match is an optimization worth mentioning. Mistake: only checking the root instead of every node. Companies: Meta.

#93. Diameter of Binary Tree — Easy · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Compute height while updating a global/closure max of `leftHeight + rightHeight` at every node. Matters as the prototype for "compute one thing (height) while tracking a different global answer" — an extremely common tree meta-pattern. Prereq: #90. Approach: helper returns height; inside it, update `maxDiameter = max(maxDiameter, left+right)` before returning `1+max(left,right)`. Insight: diameter doesn't have to pass through the root — that's exactly why you track a global max across all nodes, not just at the top call. Mistake: only checking `leftHeight+rightHeight` at the root. Companies: Meta, LinkedIn.

#94. Balanced Binary Tree — Easy · 🔵 · ⭐ ⭐ ⭐ ⭐: Height computation that returns a sentinel (e.g., -1) to short-circuit when imbalance is detected. Matters for the "encode a boolean condition inside a numeric return value to avoid a separate top-down pass" trick. Prereq: #90. Approach: helper returns height, or -1 immediately if a subtree is already unbalanced or its children are. Insight: this avoids the naive O(n²) approach of recomputing height for every node separately. Mistake: naive approach recomputing height from scratch at each node. Companies: Amazon.

#95. LCA of a Binary Search Tree — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Use BST ordering to decide direction without exploring both subtrees. Key concept: if both targets are less than current, go left; greater, go right; otherwise current node is the LCA. Matters for showing how BST structure gives O(h) instead of O(n) for LCA (contrast with #96). Prereq: BST property understanding. Approach: iterative or recursive descent using value comparisons. Insight: no need to search both subtrees — BST ordering tells you exactly which way to go. Mistake: implementing generic tree LCA (#96) and missing the BST-specific optimization. Companies: Amazon, Facebook/Meta.

#96. LCA of a Binary Tree — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: General tree LCA via post-order recursion returning "found target(s) below." Key concept: recurse into both children; if both return non-null, current node is the LCA; else propagate whichever side is non-null. Matters as one of the most-asked tree mediums, testing a subtle recursive-return-value design. Prereq: #90. Approach: `if node in (p,q) or node is None: return node`; recurse left/right; combine results. Insight: the elegance is that the function does double duty — it both searches for targets and identifies the LCA in a single pass. Mistake: needing two full traversals (compute ancestor paths separately) instead of the one-pass technique. Companies: Meta, Amazon, Google — very frequently asked.

#97. Binary Tree Right Side View — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: BFS level-order, taking the last node of every level (or DFS right-first, taking the first node seen at each depth). Matters as a level-based-thinking problem, reusing #73's BFS skeleton with a twist. Prereq: #73. Approach: BFS by level, capture the last element per level; or DFS(node, depth) visiting right child first and recording the first visit at each depth. Insight: the DFS variant is a slicker O(n) single-pass alternative worth

mentioning. Mistake: capturing the first node of each level instead of the last (wrong side). Companies: Amazon, Meta, Facebook.

#98. Count Good Nodes in Binary Tree — Medium · 🔵 · ⭐ ⭐ ⭐: DFS carrying the max value seen along the root-to-node path. Key concept: pass "maxSoFar" down the recursion; a node is "good" if its value ≥ maxSoFar. Matters for the "carry state down the recursion" pattern (as opposed to combining state up from children, as in #90/#93). Prereq: #90. Approach: DFS(node, maxSoFar), increment count if `node.val >= maxSoFar`, recurse with updated max. Insight: distinguishing "pass info down" vs "return info up" recursion shapes is a core tree-design skill. Mistake: trying to solve with only bottom-up recursion, which can't naturally track root-to-node path maxima. Companies: Amazon.

#99. Validate Binary Search Tree — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐ ⭐: DFS carrying a valid `(low, high)` range down the recursion. Key concept: each node must lie strictly within the range inherited from its ancestors, tightened at each step. Matters as a top interview question exposing a very common bug: checking only immediate children instead of the full ancestor chain. Prereq: #98 (down-recursion pattern). Approach: recurse with bounds, tighten `high` going left and `low` going right. Insight: comparing only `node.val` vs `node.left.val`/`node.right.val` is the classic wrong approach — it misses violations from grandparents further up. Mistake: exactly that local-only comparison bug. Companies: Amazon, Meta, Microsoft, Google — extremely frequently asked, and famous for hidden edge cases (duplicate values, integer overflow bounds).

#100. Kth Smallest Element in a BST — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: In-order traversal yields sorted order; stop at the kth element. Key concept: BST in-order traversal visits nodes in ascending order. Matters as the definitive application of "in-order = sorted" for BSTs. Prereq: understand traversal orders. Approach: iterative in-order traversal with an explicit stack, decrementing k until it hits 0. Insight: an iterative approach lets you early-exit at the kth node without traversing the whole tree. Follow-up: what if the BST is modified frequently (augment nodes with subtree size for O(log n) queries). Companies: Amazon, Google.

#101. Construct Binary Tree from Preorder and Inorder Traversal — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐ ⭐: Recursive construction using preorder's first element as root, then splitting inorder around it. Key concept: preorder gives roots in visiting order; inorder tells you the size/split of left vs right subtrees relative to a root. Matters as the canonical "reconstruct a tree from traversals" problem, a favorite MAANG medium. Prereq: understand traversal orders deeply. Approach: use a hash map of value→inorder-index for O(1) split lookup; recursively build using preorder pointer + inorder range. Insight: without the hash map, finding the root's index in inorder is O(n) per call, degrading to $O(n^2)$ overall — the map fixes this to O(n). Mistake: forgetting the O(n) lookup optimization, or mismanaging the shared preorder index across recursive calls. Companies: Amazon, Meta, Microsoft.

#102. Binary Tree Maximum Path Sum — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Post-order recursion returning the best *downward* path from a node, while tracking a global max that may bend through the node (left+right+val). Matters as one of the most important hard tree problems — combines #93's "track global while returning a different local value" pattern with negative-number handling. Prereq: #93, #90. Approach: helper returns `max(0, val + max(leftGain, rightGain))` (clamping negative contributions to 0); global max updated with `val + leftGain + rightGain` at every node. Insight: a path returned *upward* to the parent can only include one side (can't fork),

but the *global* answer may fork at any node — this distinction is the crux of the problem. Mistake: returning the "forked" sum upward instead of just one branch, corrupting the parent's calculation. Companies: Google, Meta — a classic hard signal question.

#103. Serialize and Deserialize Binary Tree — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Preorder DFS with explicit null markers for encoding; matching recursive parse for decoding. Matters as a top design-style hard problem testing whether you can invent a reversible encoding scheme. Prereq: #90, traversal fluency. Approach: serialize via preorder DFS, appending a sentinel (e.g., "#") for nulls, joined by commas; deserialize by consuming tokens from an iterator/queue in the same preorder order. Insight: preorder + null markers uniquely determines tree structure without needing a second traversal (unlike reconstructing from two traversals as in #101). Mistake: using an encoding that doesn't disambiguate structure (e.g., inorder alone is insufficient). Companies: Amazon, Google, Meta — a top hard-tier design question.

#104. Vertical Order Traversal of a Binary Tree — Hard · 🔴 · ⭐ ⭐ ⭐: BFS/DFS while tracking (column, row, value) triples, then sorting. Key concept: assign each node a column offset (left -1, right +1) and row depth; group by column, sort by row then value within ties. Matters as a good "combine traversal with custom multi-key sorting" hard problem. Prereq: #73. Approach: BFS collecting `(col, row, val)`, group into a map by column, sort groups by column index, sort entries within a column by `(row, val)`. Insight: the tie-breaking rule (sort by value when row and column both match) is the easy-to-miss detail that trips people up. Mistake: not handling same-row-same-column ties correctly. Companies: Amazon, Meta.

---

10. Heap / Priority Queue

Pattern Card

- **Recognition signals:** "Kth largest/smallest", "top K", "merge K sorted", "median of a stream," scheduling by priority, "closest K points."
- **Core idea:** a heap keeps the min (or max) accessible in O(1) with O(log n) insert/remove — ideal whenever you repeatedly need "the current best/worst" without full sorting. A "two heaps" (max-heap for lower half, min-heap for upper half) pattern maintains a running median.
- **Typical complexity:** O(n log k) for top-K-style problems (much better than O(n log n) full sort when $k \ll n$).
- **Common traps:** using a max-heap when a min-heap of size k is more efficient for "top K" (keep heap size capped at k, popping the smallest); forgetting most languages default to min-heaps (negate values to simulate a max-heap).

#	Problem	Difficulty	Level	Priority
105	Kth Largest Element in an Array	Medium	🔵	⭐⭐⭐⭐⭐
106	Last Stone Weight	Easy	🟢	⭐⭐⭐
107	K Closest Points to Origin	Medium	🔵	⭐⭐⭐⭐⭐
108	Kth Largest Element in a Stream	Easy	🟢	⭐⭐⭐
109	Task Scheduler	Medium	🟠	⭐⭐⭐⭐
110	Design Twitter	Medium	🟠	⭐⭐⭐
111	Find Median from Data Stream	Hard	🔴	⭐⭐⭐⭐⭐
112	Merge k Sorted Lists	Hard	🔴	⭐⭐⭐⭐⭐
113	Reorganize String	Medium	🟠	⭐⭐⭐⭐
114	Smallest Range Covering Elements from K Lists	Hard	🔴	⭐⭐

Problem Details

#105. Kth Largest Element in an Array — Medium · 🔵 · ⭐⭐⭐⭐⭐: Min-heap of size k, or Quickselect for O(n) average. Key concept: maintain a min-heap capped at size k; its root is the kth largest. Matters as the foundational "top K via bounded heap" template, and a great showcase for discussing Quickselect as an O(n) alternative. Prereq: heap basics. Approach (heap): push each element, pop if size exceeds k. Approach (Quickselect): partition like quicksort, recurse only into the side containing the target rank. Insight: Quickselect achieves O(n) average time (vs O(n log k) heap) by discarding one full side after each partition — mention both approaches for a strong answer. Mistake: sorting the whole array (O(n log n)) when a bounded heap or quickselect is clearly faster and expected. Companies: Amazon, Meta, Facebook — one of the most common heap questions.

#106. Last Stone Weight — Easy · 🟢 · ⭐⭐⭐: Max-heap simulation, repeatedly smashing the two heaviest stones. Matters as the simplest possible heap-simulation warm-up. Prereq: none. Approach: push all negated for max-heap behavior, pop two, push back the difference if nonzero. Insight: negation is the standard trick to simulate a max-heap using a min-heap library. Mistake: forgetting to re-push the difference. Companies: Google.

#107. K Closest Points to Origin — Medium · 🔵 · ⭐⭐⭐⭐⭐: Max-heap of size k keyed by distance (or Quickselect). Key concept: mirror image of #105 — keep the k *smallest* distances using a max-heap capped at size k, popping the largest when exceeded. Matters for practicing the "cap-and-pop-worst" heap template in the opposite direction from #105. Prereq: #105. Approach: push `(-distance, point)` for a max-heap simulation, pop when size > k. Insight: since you don't need exact sorted order, capping the heap at size k (not size n) is the efficiency win. Mistake: pushing all n points before trimming, instead of maintaining a live cap of k. Companies: Amazon, Facebook.

#108. Kth Largest Element in a Stream — Easy · 🟢 · ⭐ ⭐ ⭐: Persistent min-heap of size k across an online stream of `add()` calls. Matters as the "design" version of #105, testing state maintenance across multiple calls. Prereq: #105. Approach: maintain a class-level min-heap of size k; each `add()` pushes and pops if size exceeds k, returns the new root. Insight: because state persists between calls, the heap must be built incrementally, not recomputed from scratch. Mistake: rebuilding the heap on every `add()` call (defeats the purpose of streaming). Companies: Amazon.

#109. Task Scheduler — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: (see also #72) Max-heap of task counts + a cooldown queue. Matters here specifically for the heap-based simulation approach as an alternative to the closed-form formula. Prereq: #72's formula version helps but isn't required. Approach: max-heap by remaining count; pop the most frequent, decrement, push into a cooldown queue with its available-again time; refill the heap once cooldown expires. Insight: this simulation approach generalizes better than the formula to variants with different constraints. Mistake: not correctly timing when a task re-enters the heap after cooldown. Companies: Amazon, Meta.

#110. Design Twitter — Medium · 🔴 · ⭐ ⭐ ⭐: Merge-k-sorted-lists style heap over per-user tweet timelines. Key concept: to get the 10 most recent tweets across followees, push the most recent tweet from each followed user into a heap, pop-and-replace like in #112. Matters as a system-design-flavored heap application. Prereq: #112. Approach: maintain per-user tweet lists with timestamps; heap merges the most recent tweets across followed users. Insight: recognizing this as "merge k sorted lists" in disguise (each user's tweets are a sorted-by-time list) is the key insight. Mistake: sorting all tweets from all followees rather than a bounded k-way merge. Companies: Meta.

#111. Find Median from Data Stream — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Two heaps — a max-heap for the lower half, a min-heap for the upper half, kept balanced in size. Key concept: median is either the max-heap's top, the min-heap's top, or their average, depending on parity. Matters as the definitive "two heaps" pattern, a very common hard/medium-hard MAANG question. Prereq: heap fluency. Approach: insert into one heap, rebalance by moving the top element to the other heap if sizes differ by more than 1. Insight: the two-heaps invariant (`|size difference| <= 1`, max-heap's top ⩽ min-heap's top) is what makes median retrieval O(1). Mistake: letting the heaps become unbalanced, or comparing values incorrectly during rebalancing. Companies: Google, Amazon, Meta, LinkedIn — a top hard-tier design question.

#112. Merge k Sorted Lists — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: (see also #55) — listed here again as the flagship heap application connecting Linked Lists and Heap topics. Companies: Google, Amazon, Meta.

#113. Reorganize String — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Max-heap greedily placing the most frequent remaining character, ensuring no two adjacent are equal. Key concept: always place the currently most frequent character, holding the previously placed one aside for one round to avoid adjacency. Matters as a greedy+heap hybrid similar in spirit to Task Scheduler. Prereq: #109. Approach: max-heap by frequency, pop most frequent, append to result, re-push the *previous* character (held back one iteration) if it still has remaining count. Insight: this "delay re-insertion by one step" trick is the same idea used in Task Scheduler's cooldown, just cooldown=1. Mistake: not detecting the impossible case (`maxFreq > (n+1)/2`) upfront. Companies: Google, Meta.

#114. Smallest Range Covering Elements from K Lists — Hard · 🔴 · ⭐ ⭐: Min-heap over k pointers, tracking the current max across pointers, advancing the min pointer. Key concept: at each step, the range is `[currentMin, currentMax]`; advancing the smallest element's pointer is the only way to potentially shrink the range. Matters as an advanced multi-list heap problem, less common but a good stretch exercise. Prereq: #112. Approach: heap of `(value, listIndex, elementIndex)`, track running max, advance the popped list's pointer. Insight: only ever need to advance the current minimum — advancing anything else can't improve the range. Mistake: trying to consider advancing multiple pointers per step (unnecessary and incorrect). Companies: Google.

---

11. Graphs

Pattern Card

- **Recognition signals:** explicit graph/adjacency list, grid problems ("number of islands", "shortest path in a grid"), "prerequisite" / ordering problems, "connected components," anything mentioning cities/flights/dependencies.
- **Core idea:** represent the graph (adjacency list/matrix, or implicit grid), then apply BFS (shortest path in unweighted graphs, level-by-level spread), DFS (reachability, cycle detection, connected components), Union-Find (dynamic connectivity), topological sort (ordering with dependencies), or Dijkstra (weighted shortest path).
- **Typical complexity:** O(V+E) for BFS/DFS; O(E log V) for Dijkstra with a heap; O(E·α(V)) ≈ O(E) for Union-Find with path compression.
- **Common traps:** forgetting to mark nodes visited (infinite loops); using DFS for shortest-path-in-unweighted-graph instead of BFS; not detecting cycles before attempting topological sort; using plain BFS/DFS for weighted graphs where Dijkstra is required.

#	Problem	Difficulty	Level	Priority
115	Number of Islands	Medium	🔵	⭐⭐⭐⭐⭐
116	Max Area of Island	Medium	🔵	⭐⭐⭐⭐
117	Clone Graph	Medium	🔵	⭐⭐⭐⭐⭐
118	Pacific Atlantic Water Flow	Medium	🟠	⭐⭐⭐⭐
119	Surrounded Regions	Medium	🟠	⭐⭐⭐⭐
120	Course Schedule	Medium	🟠	⭐⭐⭐⭐⭐
121	Course Schedule II	Medium	🟠	⭐⭐⭐⭐
122	Graph Valid Tree	Medium	🟠	⭐⭐⭐
123	Number of Connected Components in an Undirected Graph	Medium	🟠	⭐⭐⭐⭐
124	Redundant Connection	Medium	🟠	⭐⭐⭐⭐
125	Is Graph Bipartite?	Medium	🟠	⭐⭐⭐⭐
126	Rotting Oranges	Medium	🟠	⭐⭐⭐⭐⭐
127	Word Ladder	Hard	🔴	⭐⭐⭐⭐⭐
128	Network Delay Time	Medium	🟠	⭐⭐⭐⭐
129	Cheapest Flights Within K Stops	Medium	🟠	⭐⭐⭐⭐
130	Swim in Rising Water	Hard	🔴	⭐⭐⭐
131	Alien Dictionary	Hard	🔴	⭐⭐⭐⭐
132	Min Cost to Connect All Points	Medium	🔴	⭐⭐⭐⭐
133	Reconstruct Itinerary	Hard	🔴	⭐⭐⭐

Problem Details

#115. Number of Islands — Medium · 🔵 · ⭐⭐⭐⭐⭐: DFS/BFS flood fill over a grid, counting connected components of `1`s. Matters as the single most iconic grid-graph problem — the entry point to the entire Graphs topic for most learners. Prereq: recursion/queue basics. Approach: scan the grid, on an unvisited `1` launch DFS/BFS marking the whole island visited, increment count. Insight: sinking visited land (mutate to `0`) avoids needing a separate visited matrix. Mistake: not marking visited before recursing, causing infinite loops or double counting. Variation: Number of Islands II (Union-Find, dynamic additions), Max Area of Island. Companies: Amazon, Meta, Google — extremely frequently asked.

#116. Max Area of Island — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Same flood fill as #115, returning area instead of count. Matters as direct reinforcement/variation practice. Prereq: #115. Approach: DFS returns 1 + sum of neighbor areas; track global max. Insight: the DFS "return a value" style (vs. void DFS with a counter) is a useful alternate template to know. Mistake: still using a void DFS with a global counter when a return-based DFS is cleaner here. Companies: Google.

#117. Clone Graph — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: DFS/BFS with a hash map from original node → cloned node to prevent infinite recursion on cycles. Matters as the definitive "graph traversal + cloning with cycle safety" problem. Prereq: #115. Approach: DFS from the given node; on first visit create a clone and store in map; recurse into neighbors, wiring clones together using the map to detect already-cloned nodes. Insight: the map serves double duty as both a "visited" set and the lookup for wiring correct clone references. Mistake: not checking the map before recursing, causing infinite loops on cyclic graphs. Companies: Amazon, Meta, Google.

#118. Pacific Atlantic Water Flow — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Multi-source DFS/BFS from both ocean borders, reversed flow direction, then intersect reachable sets. Key concept: instead of checking "can water from this cell reach the ocean" (expensive per-cell), reverse the question to "which cells can the ocean reach flowing uphill," starting BFS from all border cells simultaneously. Matters as a clever "reverse the direction of traversal" insight, similar in spirit to Rotting Oranges' multi-source idea. Prereq: #74 (multi-source BFS), #115. Approach: two separate multi-source BFS/DFS runs (one per ocean) using the reversed "can flow uphill into me" condition; answer is the intersection of both visited sets. Insight: reversing the traversal direction turns an O(cells × traversal) brute force into two O(cells) passes. Mistake: attempting per-cell traversal to each ocean (very slow, TLE on large grids). Companies: Google, Amazon.

#119. Surrounded Regions — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Border-first DFS/BFS marking safe regions, then flipping remaining unmarked interior `O`s. Key concept: any `O` connected to the border can never be surrounded, so first mark all border-connected `O`s as safe, then flip everything else. Matters as another "reverse the framing" grid problem — instead of checking every region for closure, identify the *complement* (border-connected) first. Prereq: #115. Approach: DFS/BFS from every border cell containing `O`, mark safe; then single pass flips all unmarked `O`→`X` and restores marked cells to `O`. Insight: starting from the border rather than trying to detect "enclosed-ness" directly avoids complex boundary-checking logic. Mistake: trying to check enclosure region-by-region instead of using the border-first trick. Companies: Amazon, Meta.

#120. Course Schedule — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Cycle detection in a directed graph (topological sort feasibility) via DFS coloring or Kahn's BFS algorithm. Key concept: a valid course ordering exists iff the prerequisite graph has no cycle. Matters as the entry point to topological sort / dependency-ordering problems, extremely common at MAANG. Prereq: #115/#117 traversal basics. Approach (DFS): 3-color nodes (unvisited/visiting/visited); a back-edge to a "visiting" node means a cycle. Approach (Kahn's BFS): repeatedly remove zero-indegree nodes; if all nodes get removed, no cycle. Insight: Kahn's algorithm elegantly gives you both cycle detection AND (in #121) the actual order for free. Mistake: using simple visited/unvisited DFS coloring (2-color) which fails to detect cycles correctly — you need the 3-state "currently in recursion stack" distinction. Companies: Amazon, Google, Meta — extremely common.

#121. Course Schedule II — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Same as #120 but return the actual topological order. Matters as direct application/output-extension of #120. Prereq: #120. Approach:

Kahn's BFS naturally produces the order as nodes are popped; DFS approach appends to a result list in post-order then reverses. Insight: DFS-based topological sort requires reversing the post-order finish sequence — an easy detail to forget. Mistake: forgetting to reverse when using the DFS approach. Companies: Amazon, Meta.

#122. Graph Valid Tree — Medium · 🔴 · ⭐ ⭐ ⭐: A graph is a valid tree iff it's connected AND has exactly `n-1` edges (no cycles). Key concept: Union-Find or DFS to check both connectivity and absence of cycles simultaneously. Matters as a clean characterization problem combining two separate graph properties into one elegant check. Prereq: Union-Find basics. Approach: if `edges.length != n-1`, immediately false; else Union-Find over all edges, checking no union ever merges already-connected nodes; verify a single connected component at the end. Insight: the edge-count check `== n-1` is a fast pre-filter before doing any traversal. Mistake: only checking edge count without verifying full connectivity (a valid tree needs both). Companies: Google, Meta.

#123. Number of Connected Components in an Undirected Graph — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Union-Find (or repeated DFS) counting the number of disjoint sets. Matters as the canonical Union-Find application, and a good side-by-side comparison with DFS-based component counting. Prereq: Union-Find basics. Approach: union every edge's endpoints; count distinct roots at the end (or track a `count` variable decremented on every successful union). Insight: path compression + union by rank/size are what make Union-Find near-O(1) per operation — always mention them. Mistake: implementing Union-Find without path compression, degrading performance on adversarial inputs. Companies: Google, Meta.

#124. Redundant Connection — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Union-Find identifying the edge that first creates a cycle. Key concept: process edges in order; the first edge whose endpoints are already connected is the redundant one. Matters as a practical Union-Find application beyond simple counting. Prereq: #123. Approach: union each edge's endpoints in order; return the edge where `find(u)==find(v)` before unioning. Insight: because the graph starts as a tree plus exactly one extra edge, the first cycle-creating edge is guaranteed unique and is the answer. Mistake: using DFS cycle detection (works but more complex than the direct Union-Find approach here). Companies: Meta.

#125. Is Graph Bipartite? — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: 2-coloring via BFS/DFS; graph is bipartite iff no edge connects two same-colored nodes. Matters as the classic 2-coloring graph problem, testable via BFS or DFS with equal ease. Prereq: #115. Approach: color each component alternately as you traverse; if a neighbor is already the same color as the current node, return false. Insight: must check every connected component separately (graph may be disconnected). Mistake: forgetting disconnected components, or not properly propagating alternating colors. Companies: Meta, Google.

#126. Rotting Oranges — (see #74, listed again as it's core to both Queue/BFS and Graphs topics.) Companies: Amazon, Google.

#127. Word Ladder — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: BFS over an implicit graph where edges connect words differing by one letter. Key concept: treat each word as a graph node; BFS finds the shortest transformation sequence since BFS naturally finds shortest paths in unweighted graphs. Matters as a flagship "recognize the implicit graph" hard problem — the graph is never given explicitly, which is the real challenge. Prereq: #115, #73 (BFS level-by-level). Approach: BFS from `beginWord`,

at each step try changing every letter position to every other letter, checking membership in the word set; level count = transformation length. Insight: precomputing wildcard patterns (`h*t` for "hot") in a map lets you find neighbors in O(word length) instead of O(dictionary size), critical for passing time limits. Mistake: naive O(dictionary size) neighbor lookup per word, causing TLE on large dictionaries. Companies: Amazon, Meta, LinkedIn — a famous hard BFS problem.

#128. Network Delay Time — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: Dijkstra's algorithm from a single source on a weighted directed graph. Matters as the canonical intro to Dijkstra, a must-know weighted-shortest-path algorithm. Prereq: #115, heap basics. Approach: min-heap of `(distance, node)`, relax edges, track visited/finalized nodes; answer is the max finalized distance (or -1 if unreachable nodes remain). Insight: Dijkstra only works correctly with non-negative weights — always state this assumption explicitly in interviews. Mistake: applying Dijkstra to a graph with negative weights (needs Bellman-Ford instead). Companies: Google, Amazon.

#129. Cheapest Flights Within K Stops — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: Modified Bellman-Ford (or Dijkstra with a stop-count dimension in state). Key concept: since Dijkstra's greedy finalization doesn't account for a "stops" constraint, use Bellman-Ford relaxation limited to `k+1` rounds, or Dijkstra with state `(cost, node, stopsUsed)`. Matters as a great example of a standard algorithm needing modification for an added constraint — testing algorithmic adaptability, not just memorization. Prereq: #128. Approach: Bellman-Ford-style — relax all edges `k+1` times using a *snapshot* of the previous round's distances (to avoid using an edge relaxed in the same round twice). Insight: standard Dijkstra's "finalize once visited" assumption breaks with a stops constraint, since a costlier-but-fewer-stops path might matter — this subtlety is the core lesson. Mistake: applying vanilla Dijkstra and getting wrong answers on the stops constraint. Companies: Google, Amazon.

#130. Swim in Rising Water — Hard · 🔴 · ⭐ ⭐ ⭐: Dijkstra-like approach (or binary search + BFS) minimizing the maximum elevation along a path. Key concept: this is a "minimize the max edge/node weight on a path" problem, solved by a modified Dijkstra using a min-heap keyed on the max-so-far value instead of cumulative sum. Matters as an advanced variant showing Dijkstra generalizes beyond simple sum-of-weights shortest paths. Prereq: #128. Approach: min-heap of `(maxElevationSoFar, row, col)`; pop smallest, update answer, push neighbors with updated max. Alternative: binary search on the answer + BFS/DFS feasibility check (reachable using only cells ⩽ candidate elevation). Insight: recognizing "minimize the maximum" problems as a Dijkstra variant (or a binary-search-on-answer candidate) is a valuable generalizable insight. Mistake: trying to use standard shortest-path summation instead of max-tracking. Companies: Google.

#131. Alien Dictionary — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Build a graph of letter-precedence constraints from adjacent word comparisons, then topological sort. Key concept: compare adjacent words in the sorted list to infer `a` comes before `b` edges; topologically sort the resulting letter graph. Matters as a hard problem combining graph construction (a non-trivial step) with topological sort (#120/#121). Prereq: #120, #121. Approach: for each adjacent word pair, find the first differing character and add a directed edge; run topological sort (Kahn's or DFS); detect invalid cases (cycle, or a word being a prefix of an earlier word which is invalid ordering). Insight: constructing the graph correctly (only from the *first* differing character between adjacent words, and validating the prefix-invalid case) is the trickiest part, not the topological sort itself. Mistake: comparing all pairs

of words instead of only adjacent ones, or missing the invalid-prefix edge case. Companies: Google, Meta, Airbnb — a well-known hard graph-construction problem.

#132. Min Cost to Connect All Points — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Minimum Spanning Tree via Prim's or Kruskal's algorithm. Key concept: connect all points with minimum total edge weight without forming cycles — the classic MST definition. Matters as the canonical MST introduction problem. Prereq: #123 (Union-Find) for Kruskal's, or heap basics for Prim's. Approach (Kruskal's): generate all edges (Manhattan distances), sort by weight, union-find to add edges without creating cycles. Approach (Prim's): min-heap growing a tree from an arbitrary start node, always adding the cheapest edge to an unvisited node. Insight: Prim's is usually faster here since the graph is dense (all pairs of points are potential edges — $O(n^2)$ edges), making Kruskal's sort step expensive; Prim's with a heap avoids ever materializing all edges explicitly. Mistake: using Kruskal's naively on a dense graph without considering the edge-generation cost. Companies: Google, Meta.

#133. Reconstruct Itinerary — Hard · 🔴 · ⭐ ⭐ ⭐: Eulerian-path-style DFS (Hierholzer's algorithm) using all edges exactly once, sorted lexicographically. Key concept: greedily visit the lexicographically smallest next airport, backtracking (via post-order append) when stuck to guarantee all edges are used. Matters as an advanced graph-traversal problem requiring awareness of Eulerian paths, less common but a strong differentiator. Prereq: #115, #120. Approach: build adjacency lists sorted lexicographically (as min-heaps or sorted lists), DFS using each edge once, appending nodes to the result in *post-order*, then reverse. Insight: the "append in post-order, then reverse" trick is what correctly handles dead-ends requiring backtracking — this is exactly Hierholzer's algorithm for Eulerian paths. Mistake: trying a purely greedy forward DFS without post-order reversal, which fails on itineraries requiring backtracking. Companies: Google.

---

12. Dynamic Programming

Pattern Card

- **Recognition signals:** "count the number of ways," "min/max cost to reach," "can you partition/reach a target," overlapping subproblems visible in a naive recursive solution, optimal substructure ("the best solution to the whole uses the best solution to a smaller piece").
- **Core idea:** define a recurrence relating a subproblem to smaller subproblems, memoize (top-down) or tabulate (bottom-up) to avoid recomputation. The real skill is *defining the state* (what varies?) and the *transition* (how does one state relate to others?) — not the coding itself.
- **Typical complexity:** O(states × transition cost) — commonly $O(n)$, $O(n^2)$, or $O(n \cdot k)$ depending on state dimensions.
- **Common traps:** under-specifying state (missing a dimension the recurrence actually depends on); off-by-one in base cases; not recognizing a greedy-looking problem actually requires DP because greedy fails on counterexamples; recomputing subproblems without memoization (exponential blowup).

#	Problem	Difficulty	Level	Priority
134	Climbing Stairs	Easy	🟢	⭐⭐⭐⭐⭐
135	House Robber	Medium	🔵	⭐⭐⭐⭐⭐
136	House Robber II	Medium	🟠	⭐⭐⭐⭐
137	Decode Ways	Medium	🟠	⭐⭐⭐⭐
138	Coin Change	Medium	🔵	⭐⭐⭐⭐⭐
139	Coin Change II	Medium	🟠	⭐⭐⭐⭐
140	Maximum Product Subarray	Medium	🟠	⭐⭐⭐⭐
141	Word Break	Medium	🟠	⭐⭐⭐⭐⭐
142	Longest Increasing Subsequence	Medium	🟠	⭐⭐⭐⭐⭐
143	Partition Equal Subset Sum	Medium	🟠	⭐⭐⭐⭐⭐
144	Target Sum	Medium	🟠	⭐⭐⭐
145	Combination Sum IV	Medium	🟠	⭐⭐⭐
146	Unique Paths	Medium	🔵	⭐⭐⭐⭐⭐
147	Minimum Path Sum	Medium	🔵	⭐⭐⭐⭐
148	Longest Common Subsequence	Medium	🟠	⭐⭐⭐⭐⭐
149	Edit Distance	Hard	🔴	⭐⭐⭐⭐⭐
150	Longest Palindromic Substring	Medium	🟠	⭐⭐⭐⭐⭐
151	Palindromic Substrings	Medium	🟠	⭐⭐⭐⭐
152	Interleaving String	Medium	🔴	⭐⭐⭐
153	Distinct Subsequences	Hard	🔴	⭐⭐⭐
154	Best Time to Buy and Sell Stock with Cooldown	Medium	🔴	⭐⭐⭐⭐
155	Best Time to Buy and Sell Stock III	Hard	🔴	⭐⭐⭐⭐
156	Best Time to Buy and Sell Stock IV	Hard	🔴	⭐⭐⭐
157	Burst Balloons	Hard	🔴	⭐⭐⭐⭐
158	Regular Expression Matching	Hard	🔴	⭐⭐⭐⭐

#	Problem	Difficulty	Level	Priority
159	Maximal Square	Medium	🔴	⭐⭐⭐⭐
160	Unique Binary Search Trees	Medium	🔴	⭐⭐⭐

Problem Details

#134. Climbing Stairs — Easy · 🟢 · ⭐⭐⭐⭐⭐: `dp[i]=dp[i-1]+dp[i-2]` (it's Fibonacci). Matters as the simplest possible 1D DP, teaching state definition (`dp[i]` = ways to reach step i) and transition. Prereq: none. Approach: bottom-up array or two rolling variables. Insight: recognizing this is structurally identical to Fibonacci — many DP problems reduce to well-known sequences. Mistake: not seeing that O(1) space (two variables) suffices instead of a full array. Companies: Amazon, Adobe.

#135. House Robber — Medium · 🔵 · ⭐⭐⭐⭐⭐: `dp[i]=max(dp[i-1], dp[i-2]+nums[i])` — take or skip. Matters as the template "take vs skip with adjacency constraint" DP, reused everywhere. Prereq: #134. Approach: rolling two-variable DP tracking "best if we don't take current" and "best if we do." Insight: this "include/exclude with a gap constraint" recurrence generalizes to many scheduling-style DPs. Mistake: trying greedy (always skip alternating houses) — fails on many inputs; must use real DP. Companies: Amazon, LinkedIn.

#136. House Robber II — Medium · 🔴 · ⭐⭐⭐⭐: Circular array variant — run #135 twice, excluding first or last house respectively. Matters for teaching "reduce a harder circular constraint to two runs of a known linear solution." Prereq: #135. Approach: `max(rob(houses[0:n-1]), rob(houses[1:n]))`. Insight: excluding one endpoint each time correctly handles the "first and last are adjacent" circular constraint without new DP logic. Mistake: trying to write new circular-aware DP recurrence instead of the two-runs reduction. Companies: Amazon.

#137. Decode Ways — Medium · 🔴 · ⭐⭐⭐⭐: `dp[i] = dp[i-1] (if single digit valid) + dp[i-2] (if two-digit valid)`. Matters as a string-DP problem testing careful validity-condition handling (leading zeros, valid two-digit range 10-26). Prereq: #134. Approach: bottom-up array, checking single and double-digit validity at each position. Insight: this recurrence shape mirrors Climbing Stairs exactly — recognizing "decode ways == climbing stairs with validity gating" is the transferable insight. Mistake: mishandling '0' (only valid as the second digit of 10 or 20). Companies: Meta, Amazon.

#138. Coin Change — Medium · 🔵 · ⭐⭐⭐⭐⭐: Unbounded knapsack — `dp[amount] = min(dp[amount-coin]+1)` over all coins. Matters as the definitive unbounded-knapsack template (coins reusable), contrasted with 0/1 knapsack. Prereq: #134. Approach: bottom-up array sized `amount+1`, initialize to infinity except `dp[0]=0`, iterate amounts and coins. Insight: looping coins inside the amount loop (or vice versa) doesn't matter for the *minimum* variant, but *does* matter for the *counting* variant (#139) — a subtlety worth explicitly understanding. Mistake: not initializing unreachable amounts to infinity, causing incorrect min comparisons. Companies: Amazon, Meta, Uber — one of the most-asked DP mediums.

#139. Coin Change II — Medium · 🔴 · ⭐⭐⭐⭐: Unbounded knapsack, counting combinations (not permutations). Key concept: loop coins in the *outer* loop and amount in the *inner* loop — this

ordering ensures each combination is counted once regardless of coin order. Matters as the critical companion to #138, teaching that loop order changes the *meaning* of the DP (combinations vs permutations). Prereq: #138. Approach: `dp[amt] += dp[amt-coin]`, coins outer, amounts inner. Insight: swapping the loop order (amount outer, coin inner) instead computes *permutations* (Combination Sum IV, #145) — same recurrence shape, different semantics from loop order alone. Mistake: not realizing loop order matters here, unlike in #138. Companies: Google.

#140. Maximum Product Subarray — (see #12, listed again as it's core 1D DP tracking max AND min state.) Companies: Amazon, LinkedIn.

#141. Word Break — Medium · 🔴 · ⭐⭐⭐⭐⭐: `dp[i] = true if any j<i has dp[j]==true and s[j:i] in wordDict`. Matters as the template for "can the string be segmented" DP, and a bridge from Recursion/Backtracking (#84) into DP via memoization. Prereq: #84 conceptually, #134. Approach: bottom-up boolean array, `dp[0]=true`, check every valid split point. Insight: a naive recursive-backtracking solution without memoization is exponential; adding a memo table (top-down) or converting to bottom-up DP is what makes it polynomial. Mistake: forgetting memoization in a recursive attempt, causing TLE. Companies: Amazon, Meta, Google — extremely common.

#142. Longest Increasing Subsequence — Medium · 🔴 · ⭐⭐⭐⭐⭐: O(n²) DP (`dp[i]=1+max(dp[j]) for j<i, nums[j]<nums[i]`), or O(n log n) via patience sorting with binary search. Matters as one of the most important DP problems, testing whether you know the classic O(n log n) optimization beyond brute DP. Prereq: #134, #35 (binary search) for the optimized version. Approach (O(n²)): straightforward DP. Approach (O(n log n)): maintain a "tails" array of smallest tail for each subsequence length, binary search for the insertion point on each new number. Insight: the `tails` array itself isn't a valid LIS, but its *length* always equals the LIS length — a subtle but crucial realization. Mistake: believing the tails array must literally be the LIS. Variation: Number of LIS, Russian Doll Envelopes (2D LIS), Longest Chain of Pairs. Companies: Google, Microsoft, Meta — a top hard-ish medium.

#143. Partition Equal Subset Sum — Medium · 🔴 · ⭐⭐⭐⭐⭐: 0/1 knapsack — can a subset sum to `totalSum/2`? Matters as the canonical 0/1 knapsack (each item used at most once) contrasted against #138's unbounded knapsack. Prereq: #138. Approach: boolean DP array over achievable sums, iterate items in outer loop, sums in *descending* inner loop to avoid reusing an item twice in the same pass. Insight: iterating the sum dimension in descending order is exactly what enforces the "0/1" (use-once) constraint when using a 1D rolling array. Mistake: iterating sums ascending, which incorrectly allows reusing the same item multiple times (turns it into unbounded knapsack). Companies: Meta, Amazon.

#144. Target Sum — Medium · 🔴 · ⭐⭐⭐: Reduce +/- sign assignment to a subset-sum problem: `P - N = target, P + N = totalSum ⟹ P = (target+totalSum)/2`. Matters for teaching "transform the problem into a known template" (0/1 knapsack subset counting) rather than a totally new DP. Prereq: #143. Approach: count subsets summing to `P` using a 0/1 knapsack counting DP. Insight: the algebraic transformation from a sign-assignment problem into a subset-sum-counting problem is the entire difficulty — the DP itself is standard once transformed. Mistake: attempting brute-force backtracking over all 2^n sign assignments without recognizing the transformation (works for small n but not the intended solution). Companies: Meta.

#145. Combination Sum IV — Medium · 🔴 · ⭐ ⭐ ⭐: Counting *permutations* (order matters) reaching a target — loop amount outer, coins inner (opposite of #139). Matters as the direct contrast case proving loop-order changes DP semantics. Prereq: #139. Approach: `dp[amt] += dp[amt-coin]` for each coin, amount as outer loop. Insight: despite an identical-looking recurrence to #139, swapping loop order changes "combinations" into "permutations" — a must-internalize subtlety. Mistake: confusing this with #139 and getting the loop order backwards for the intended semantics. Companies: Google.

#146. Unique Paths — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`. Matters as the canonical 2D grid DP, and also solvable via combinatorics (`C(m+n-2, m-1)`) — knowing both is a strong signal. Prereq: #134. Approach: 2D table or rolling 1D array, first row/col initialized to 1. Insight: this is literally Pascal's Triangle in disguise, connecting DP to combinatorics. Mistake: not recognizing the combinatorial closed form as a faster O(m+n) alternative to O(mn) DP. Companies: Amazon, Bloomberg.

#147. Minimum Path Sum — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`. Matters as a direct extension of #146 to a cost-minimization variant. Prereq: #146. Approach: 2D or rolling 1D array; base row/column accumulate directly. Insight: the technique of collapsing a 2D DP into a 1D rolling array (updating in place, left to right, top to bottom) saves space and is worth explicitly mentioning. Mistake: not handling the first row/column's special-case initialization. Companies: Amazon.

#148. Longest Common Subsequence — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: `dp[i][j] = dp[i-1][j-1]+1 if chars match else max(dp[i-1][j], dp[i][j-1])`. Matters as THE foundational 2-string DP template, underlying Edit Distance, Longest Common Substring, Shortest Common Supersequence, and more. Prereq: #146 (2D DP grid mental model). Approach: 2D table indexed by prefixes of both strings. Insight: the "match → diagonal+1, else → best of left/up" recurrence shape is the master template for the entire two-string DP family. Mistake: confusing "subsequence" (non-contiguous, order preserved) with "substring" (contiguous) — a completely different, simpler DP. Companies: Amazon, Meta, Google, Microsoft — a cornerstone question.

#149. Edit Distance — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: `dp[i][j] = dp[i-1][j-1] if match, else 1+min(insert, delete, replace)`. Matters as the direct generalization of #148 to three operations, one of the most iconic hard DP problems industry-wide. Prereq: #148. Approach: 2D table; `dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1] + (0 if match else 1))`. Insight: each of the three DP neighbors corresponds exactly to one edit operation (delete, insert, replace) — visualizing this correspondence is what makes the recurrence memorable rather than memorized by rote. Mistake: mixing up which neighbor corresponds to insert vs delete (harmless for the final answer since it's symmetric under relabeling, but important to explain correctly). Companies: Google, Meta, Amazon, Microsoft — an extremely famous hard problem.

#150. Longest Palindromic Substring — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: DP (`dp[i][j] = dp[i+1][j-1] and s[i]==s[j]`), or the more efficient "expand around center" $O(n^2)$/$O(1)$-space technique. Matters as the classic contiguous-palindrome problem, contrasted with #148's subsequence DP. Prereq: understanding palindromes. Approach (expand around center): for every index (and every gap between indices, for even-length palindromes), expand outward while characters match. Insight: "expand around center" achieves the same $O(n^2)$ time as full DP but with $O(1)$ space

instead of O(n²) — usually the preferred interview answer. Mistake: forgetting to check both odd-length (single center) and even-length (center between two indices) cases. Companies: Amazon, Meta, Microsoft — one of the most-asked string problems overall.

#151. Palindromic Substrings — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Same "expand around center" technique as #150, counting instead of finding the longest. Matters as direct reinforcement of the center-expansion technique. Prereq: #150. Approach: expand around every center (2n-1 total centers), increment count for every valid palindrome found. Insight: recognizing this reuses the *exact* same expansion routine as #150, only the aggregation changes (count vs track-longest). Mistake: re-deriving from scratch instead of reusing the #150 routine. Companies: Meta, Amazon.

#152. Interleaving String — Medium · 🔴 · ⭐ ⭐ ⭐: 2D DP over prefixes of two source strings, checking if the interleaving is achievable. Key concept: `dp[i][j] = (dp[i-1][j] and s1[i-1]==s3[i+j-1]) or (dp[i][j-1] and s2[j-1]==s3[i+j-1])`. Matters as a less common but instructive 2-string DP variant, good for showing depth in the two-string DP family. Prereq: #148. Approach: 2D boolean table over prefix lengths of s1 and s2. Insight: length pre-check (`len(s1)+len(s2)==len(s3)`) is a fast early rejection before running the full DP. Mistake: not doing the fast length pre-check, or misindexing `s3` position as `i+j-1`. Companies: Google.

#153. Distinct Subsequences — Hard · 🔴 · ⭐ ⭐ ⭐: Counting DP over two strings — `dp[i][j] = dp[i-1][j-1]+dp[i-1][j] if match else dp[i-1][j]`. Matters as a counting-variant of the LCS family, testing whether you can adapt the two-string DP template from boolean/max to counting semantics. Prereq: #148. Approach: 2D table counting ways to form `t[0:j]` from `s[0:i]` as a subsequence. Insight: on a character match, you have *two* choices (use this match, or skip it and still look for a match later) — both must be added, not just the matched branch. Mistake: only taking the matched branch and ignoring the "skip this character of s" branch even when characters match. Companies: Google, Meta.

#154. Best Time to Buy and Sell Stock with Cooldown — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: State-machine DP with three states (held, sold-today, resting/cooldown). Matters as the definitive "state machine" DP, generalizing the simple stock problems into a small finite-state model. Prereq: #26, #135. Approach: track `hold`, `sold`, `rest` states per day, transition based on buy/sell/cooldown rules. Insight: drawing the state-transition diagram explicitly (hold→sold→rest→hold...) before coding is the key technique for all "stock with constraint" variants. Mistake: trying ad hoc conditionals instead of formalizing distinct states. Companies: Amazon, Meta.

#155. Best Time to Buy and Sell Stock III — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: DP with a transaction-count dimension (at most 2 transactions). Key concept: track 4 states — bought1, sold1, bought2, sold2 — updated greedily in one pass. Matters as a "DP with an extra state dimension for a count limit" template, generalized further in #156. Prereq: #154. Approach: single pass updating `buy1=max(buy1,-price)`, `sell1=max(sell1,buy1+price)`, `buy2=max(buy2,sell1-price)`, `sell2=max(sell2,buy2+price)`. Insight: processing all four states in this specific order within the same loop iteration correctly chains dependencies within a single day. Mistake: wrong update order causing a transaction to "use" the same day's price twice inconsistently. Companies: Meta, Amazon.

#156. Best Time to Buy and Sell Stock IV — Hard · 🔴 · ⭐ ⭐ ⭐: Generalizes #155 to at most `k` transactions using a 2D DP over (transaction count, day). Matters as the fully general version,

reinforcing that #155's 4-variable trick is really `k=2` unrolled. Prereq: #155. Approach: `dp[t][d] = max(dp[t][d-1], price[d] + max(dp[t-1][d'] - price[d']) for d'<d)`, optimized by tracking a running max of `dp[t-1][d']-price[d']`. Insight: when `k >= n/2`, the problem reduces to unlimited transactions (simple greedy, sum all positive differences) — an important special-case optimization. Mistake: not handling the large-k special case, causing unnecessary O(nk) computation when O(n) greedy suffices. Companies: Google.

#157. Burst Balloons — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: Interval DP — `dp[l][r]` = max coins from bursting all balloons strictly between l and r, choosing the *last* balloon to burst in that range. Key concept: thinking about which balloon is burst *last* (not first) within a range makes neighbors well-defined throughout the process. Matters as the definitive interval DP problem and a favorite hard MAANG question testing a genuinely counter-intuitive reformulation. Prereq: #148 for 2D DP comfort. Approach: pad the array with 1s at both ends; `dp[l][r] = max over k in (l,r) of dp[l][k]+dp[k][r]+nums[l]*nums[k]*nums[r]`, iterating by increasing interval length. Insight: choosing "last burst in range" instead of "first burst" is the reformulation that makes the subproblems independent — a genuinely surprising insight worth deeply understanding, not memorizing. Mistake: trying to model "first balloon burst," which creates dependency between overlapping subproblems and doesn't decompose cleanly. Companies: Google, LinkedIn.

#158. Regular Expression Matching — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐: 2D DP over pattern and text, with special handling for `.` and `*`. Key concept: `*` means "zero or more of the preceding element," requiring a DP transition that considers both "use the * to match nothing" and "use it to match one more occurrence." Matters as one of the hardest string DP problems, frequently used to separate strong candidates. Prereq: #148, #149. Approach: `dp[i][j]` = does `s[0:i]` match `p[0:j]`; special-case `p[j-1]=='*'` to consider `dp[i][j-2]` (zero occurrences) or `dp[i-1][j]` (one more occurrence, if current chars match). Insight: `*` always pairs with the character *before* it in the pattern — handling this pairing correctly (not treating `*` in isolation) is the crux. Mistake: mishandling the "zero occurrences" branch of `*`, a very common source of bugs. Companies: Google, Meta, Uber — a top-tier hard problem.

#159. Maximal Square — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: `dp[i][j] = 1+min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])` if `matrix[i][j]==1`. Key concept: the DP value represents the side length of the largest square with bottom-right corner at (i,j), bounded by the *minimum* of its three neighbors (a square can't be bigger than its tightest constraint). Matters as a 2D DP problem with a clever "min of neighbors" recurrence, distinct from the more common "max of neighbors" shape. Prereq: #146. Approach: 2D DP table, track the running max value seen (which is the side length of the answer). Insight: the "min, not max" of neighboring DP values is the surprising twist that trips people up if they pattern-match too quickly from other grid DPs. Mistake: defaulting to max() by habit from other grid DP problems. Companies: Google, Amazon.

#160. Unique Binary Search Trees — Medium · 🔴 · ⭐ ⭐ ⭐: Catalan number recurrence — `dp[n] = sum(dp[i]*dp[n-1-i]) for i in 0..n-1`. Key concept: for each choice of root value, the left subtree uses smaller values and the right subtree uses larger ones, and the count of valid trees is independent of *which* specific values, only *how many*. Matters as an elegant combinatorics-meets-DP problem, and an entry point to the Catalan number family (also appears in valid parentheses counting, polygon triangulation). Prereq: #134. Approach: bottom-up DP over `n`, using the recurrence above. Insight: recognizing the answer only depends on subtree *size*, not the actual

value set, is what collapses an apparently combinatorial-explosion problem into simple O(n²) DP. Mistake: trying to enumerate actual trees (that's Unique BST II — exponential, different problem) instead of just counting. Companies: Amazon, Bloomberg.

---

13. Greedy

Pattern Card

- **Recognition signals:** "minimum number of X to achieve Y," interval scheduling, "can you reach the end," problems where a locally-optimal choice provably never hurts the global answer.
- **Core idea:** make the locally best choice at each step and never reconsider it — valid only when you can *prove* an exchange argument (swapping any other choice for the greedy one never makes things worse). Sorting is almost always step 1.
- **Typical complexity:** O(n log n) (dominated by the initial sort).
- **Common traps:** applying greedy without a valid exchange-argument proof (many greedy-*looking* problems actually require DP, e.g. 0/1 knapsack); picking the wrong sort key (by start vs by end time matters enormously in interval problems).

#	Problem	Difficulty	Level	Priority
161	Maximum Subarray	Medium	🔵	⭐⭐⭐⭐⭐
162	Jump Game	Medium	🔵	⭐⭐⭐⭐⭐
163	Jump Game II	Medium	🟠	⭐⭐⭐⭐⭐
164	Gas Station	Medium	🟠	⭐⭐⭐⭐
165	Hand of Straights	Medium	🟠	⭐⭐⭐
166	Merge Triplets to Form Target Triplet	Medium	🟠	⭐⭐⭐
167	Partition Labels	Medium	🔵	⭐⭐⭐⭐⭐
168	Valid Parenthesis String	Medium	🟠	⭐⭐⭐⭐
169	Non-overlapping Intervals	Medium	🔵	⭐⭐⭐⭐⭐
170	Candy	Hard	🔴	⭐⭐⭐

Problem Details

#161. Maximum Subarray — (see #11, revisited here as a greedy-vs-DP dual-lens problem: Kadane's is both a valid 1D DP and a valid greedy "restart when negative" argument.) Companies: Amazon, LinkedIn.

#162. Jump Game — Medium · 🔵 · ⭐⭐⭐⭐⭐: Greedy tracking the farthest reachable index. Key concept: iterate positions; if the current index exceeds the farthest reachable so far, it's unreachable — fail; otherwise update farthest = `max(farthest, i+nums[i])`. Matters as the entry

point to greedy-on-reachability problems, initially looking like it needs DP/BFS but solvable in O(n) with a single greedy pass. Prereq: none. Approach: single forward pass maintaining `maxReach`. Insight: you never need to track *which* jumps were made — only the frontier of reachability matters. Mistake: attempting a DP or BFS solution (correct but $O(n^2)$/slower) when a simple O(n) greedy pass suffices. Companies: Amazon, Meta.

#163. Jump Game II — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Greedy BFS-like "level" expansion — track current jump's boundary and the farthest reachable within the next jump. Key concept: this is implicitly BFS level-order traversal but computed greedily without an explicit queue. Matters as a step up from #162, now minimizing jump count, a more subtle greedy proof. Prereq: #162. Approach: maintain `currentEnd` (boundary of current jump) and `farthest`; when `i==currentEnd`, increment jumps and set `currentEnd=farthest`. Insight: recognizing the analogy to BFS levels (each "jump" = one BFS level) clarifies why incrementing exactly at the boundary is correct. Mistake: incrementing the jump counter at the wrong index (too early or late relative to the boundary). Companies: Amazon, Meta, Google.

#164. Gas Station — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Greedy single-pass with a key insight: if total gas ⩾ total cost, a valid start exists, and it can be found in one linear scan by resetting the candidate start whenever the running tank goes negative. Matters for teaching a non-obvious greedy exchange argument (if you can't reach station j from i, you also can't reach it from any station between i and j). Prereq: #162 comfort with greedy proofs. Approach: track `total` and `tank`; whenever `tank<0`, reset `start=i+1` and `tank=0`. Insight: the "no station strictly between the failed start and the failure point can be a valid start either" argument is the exchange proof that licenses skipping ahead directly to `i+1`. Mistake: trying to check every starting station individually ($O(n^2)$) instead of the O(n) single-pass trick. Companies: Meta, Uber.

#165. Hand of Straights — Medium · 🔴 · ⭐ ⭐ ⭐: Greedy + frequency map — always try to build a straight starting from the smallest remaining card. Matters as a frequency-map-plus-greedy hybrid, reinforcing "always resolve the smallest/leftmost unresolved element first." Prereq: #4 (frequency maps). Approach: count frequencies, repeatedly take the smallest available card and consume `groupSize` consecutive values from it. Insight: always starting from the current minimum is provably correct — any straight not starting at the minimum leaves the minimum stranded, unable to be used later. Mistake: not processing cards in increasing order, leaving impossible-to-use leftover cards. Companies: Meta.

#166. Merge Triplets to Form Target Triplet — Medium · 🔴 · ⭐ ⭐ ⭐: Greedy filtering — only consider triplets that don't exceed the target in any coordinate, then check if the max achievable in each coordinate reaches target. Matters as a simple but non-obvious greedy filter-then-check pattern. Prereq: none. Approach: discard any triplet with a coordinate exceeding target; track the running max per coordinate among remaining triplets; check if it equals target in all three. Insight: a triplet exceeding target in any single coordinate can never help (merging only takes max, never reduces) — this filtering step is the entire insight. Mistake: not discarding invalid triplets first, corrupting the max tracking. Companies: Meta.

#167. Partition Labels — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Greedy interval merging using each character's last-occurrence index. Key concept: extend the current partition's boundary to the last occurrence of every character seen so far within it; close the partition once you reach that boundary. Matters as an elegant greedy problem disguised as a string problem — it's really interval

merging (#169's family) once you compute last-occurrence indices. Prereq: #4 (hashing), #169 conceptually. Approach: precompute last index of each character; scan, extending `end = max(end, lastIndex[char])`; when `i==end`, close a partition. Insight: recognizing the reduction to "merge overlapping character-span intervals" is the key transferable insight. Mistake: not precomputing last occurrences first, leading to incorrect boundary extension. Companies: Meta, Amazon.

#168. Valid Parenthesis String — Medium · 🔴 · ⭐ ⭐ ⭐ ⭐: Greedy range-tracking — maintain a `[lowOpen, highOpen]` range of possible open-parenthesis counts, since `*` can be `(`, `)`, or empty. Matters as a clever greedy alternative to a DP/backtracking solution, tracking a *range* of possibilities instead of exact state. Prereq: #58 (stack intuition helps conceptually). Approach: iterate characters, adjust `low`/`high` bounds per character type, clamp `low` to 0; fail early if `high<0`; success if `low==0` at the end. Insight: tracking a range of feasible counts (rather than branching into all `3^n` possibilities) collapses an apparently exponential problem into O(n) greedy. Mistake: trying full backtracking/DP over all wildcard assignments (correct but far less efficient than the intended greedy solution). Companies: Adobe, Meta.

#169. Non-overlapping Intervals — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐ ⭐: Greedy — sort by end time, keep an interval if it starts after the last kept interval's end, else remove it. Matters as the canonical interval-scheduling greedy problem underlying nearly all interval problems. Prereq: sorting fluency. Approach: sort by end time, greedily keep non-overlapping intervals, count removals for the rest. Insight: sorting by *end* time (not start time) is what makes the greedy choice provably optimal — this is the classic "activity selection" exchange argument. Mistake: sorting by start time instead, which does not guarantee optimal greedy selection. Companies: Google, Meta, LinkedIn.

#170. Candy — Hard · 🔴 · ⭐ ⭐ ⭐: Two-pass greedy — left-to-right pass ensures higher-rated-than-left-neighbor gets more candy, right-to-left pass ensures the same relative to the right neighbor, taking the max of both passes per child. Matters as a harder greedy problem requiring two directional passes combined, rather than one obvious single pass. Prereq: #167 comfort with greedy scanning. Approach: initialize all to 1 candy; left pass increments if `ratings[i]>ratings[i-1]`; right pass increments (taking max with existing value) if `ratings[i]>ratings[i+1]`. Insight: a single pass in only one direction cannot satisfy both neighbor constraints simultaneously — two passes combined via max is required. Mistake: attempting a single-pass solution, which fails on inputs like strictly decreasing-then-increasing sequences. Companies: Google.

---

14. Intervals

Pattern Card

- **Recognition signals:** arrays of `[start, end]` pairs, "merge overlapping," "insert a new meeting," "minimum rooms needed," calendar/scheduling wording.
- **Core idea:** sort intervals by start (for merging) or track boundaries with a sweep-line (for counting overlaps at any point). Two intervals overlap iff `a.start < b.end and b.start < a.end`.
- **Typical complexity:** O(n log n), dominated by sorting.
- **Common traps:** sorting by the wrong key (start vs end depends on the problem); off-by-one on whether touching intervals (`[1,2],[2,3]`) count as overlapping; not merging in place efficiently.

#	Problem	Difficulty	Level	Priority
171	Merge Intervals	Medium	🔵	⭐⭐⭐⭐⭐
172	Insert Interval	Medium	🔵	⭐⭐⭐⭐⭐
173	Non-overlapping Intervals	Medium	🔵	⭐⭐⭐⭐⭐
174	Meeting Rooms	Easy	🟢	⭐⭐⭐⭐
175	Meeting Rooms II	Medium	🔴	⭐⭐⭐⭐⭐
176	Minimum Interval to Include Each Query	Hard	🔴	⭐⭐⭐

Problem Details

#171. Merge Intervals — Medium · 🔵 · ⭐⭐⭐⭐⭐: Sort by start, merge whenever the current interval's start ⩽ the last merged interval's end. Matters as the foundational interval-merging template underlying most of this topic. Prereq: sorting. Approach: sort, iterate, extend the last merged interval's end if overlapping, else start a new one. Insight: sorting by start is what guarantees you only ever need to compare against the *most recently merged* interval, never look further back. Mistake: forgetting to sort first, or comparing against the wrong "last" interval. Companies: Amazon, Meta, Google, Microsoft — extremely frequently asked.

#172. Insert Interval — Medium · 🔵 · ⭐⭐⭐⭐⭐: Three-phase scan — add all intervals ending before the new one starts, merge all overlapping intervals into the new one, add the remainder. Matters as a variant of #171 for an already-sorted list plus one new insertion, testable in a single O(n) pass without re-sorting. Prereq: #171. Approach: linear scan with three clearly separated phases (before/overlap/after). Insight: since input is already sorted, you never need to re-sort — a full O(n log n) merge-intervals-from-scratch is a weaker answer than the O(n) three-phase scan. Mistake: falling back to append-then-sort-then-merge, missing the more efficient direct approach. Companies: Google, LinkedIn.

#173. Non-overlapping Intervals — (see #169, listed again as it belongs equally to Greedy and Intervals.) Companies: Google, Meta.

#174. Meeting Rooms — Easy · 🟢 · ⭐⭐⭐⭐: Sort by start, check any adjacent pair overlaps. Matters as the simplest possible "can one person attend all meetings" check, a prerequisite mental model for #175. Prereq: sorting. Approach: sort by start time, check `intervals[i].start < intervals[i-1].end` for any i. Insight: sorting collapses an $O(n^2)$ pairwise overlap check into a single O(n) adjacent-pair scan. Mistake: checking all pairs instead of just adjacent ones after sorting. Companies: Meta, Google (often as a warm-up before #175).

#175. Meeting Rooms II — Medium · 🔴 · ⭐⭐⭐⭐⭐: Two-pointer sweep on separately sorted start and end times, or a min-heap of end times. Key concept: track how many meetings are simultaneously in progress — the maximum concurrent count is the answer. Matters as one of the most iconic scheduling problems in interviews, testable via two clean approaches (sweep line, or heap). Prereq: #174. Approach (sweep line): sort starts and ends separately; two pointers — if next start < next end, a new room is needed (increment counter), else a room frees up (decrement);

track the max. Approach (heap): min-heap of end times; if the earliest-ending meeting ends before the current one starts, reuse that room (pop+push), else push a new room. Insight: separating starts and ends into their own sorted arrays for the sweep-line approach is what makes O(n log n) two-pointer counting possible without an explicit heap. Mistake: only tracking total rooms ever used instead of the maximum *concurrent* usage. Companies: Amazon, Meta, Google, Facebook — a top scheduling question.

#176. Minimum Interval to Include Each Query — Hard · 🔴 · ⭐ ⭐ ⭐: Sort intervals by start and queries by value, use a min-heap of (interval size, end) to always have the smallest valid interval at the top. Key concept: process queries in increasing order, adding all intervals that have "started" by that query's value to the heap, and popping any that have already "ended." Matters as an advanced combination of sorting, offline query processing, and heaps. Prereq: #175, #107 (heap fluency). Approach: sort both intervals and queries; sweep query values in order, pushing newly-valid intervals to a heap keyed by size, popping expired ones, answer = heap top's size (or -1 if empty). Insight: processing queries in sorted order (an "offline" technique) rather than per-query independently is what makes this efficient overall. Mistake: processing queries in their original given order instead of sorted order, losing the incremental sweep property. Companies: Google.

---

15. Bit Manipulation

Pattern Card

- **Recognition signals:** "without using +/-", "find the single/missing number," "count set bits," subsets via bitmask, "power of two/four" checks, XOR-heavy wording.
- **Core idea:** XOR cancels identical values (`a^a=0`, `a^0=a`) making it ideal for "find the odd one out" problems; `n & (n-1)` clears the lowest set bit (useful for counting bits or checking powers of two); a bitmask can represent a subset of up to ~20-ish items compactly for DP-over-subsets.
- **Typical complexity:** O(1) per operation, O(32) or O(64) for full-integer bit scans, O(n·2^n) for bitmask DP over n items.
- **Common traps:** sign-extension issues with right shifts on negative numbers in some languages; off-by-one when iterating bit positions; forgetting `n & (n-1) == 0` also incorrectly flags `n=0` as a power of two (needs an explicit `n>0` check).

#	Problem	Difficulty	Level	Priority
177	Single Number	Easy	🟢	⭐⭐⭐⭐⭐
178	Number of 1 Bits	Easy	🟢	⭐⭐⭐⭐
179	Counting Bits	Easy	🔵	⭐⭐⭐⭐
180	Reverse Bits	Easy	🟢	⭐⭐⭐
181	Missing Number	Easy	🟢	⭐⭐⭐⭐
182	Sum of Two Integers	Medium	🟠	⭐⭐⭐
183	Single Number II	Medium	🟠	⭐⭐⭐
184	Single Number III	Medium	🟠	⭐⭐⭐
185	Subsets	Medium	🔵	⭐⭐⭐

Problem Details

#177. Single Number — Easy · 🟢 · ⭐⭐⭐⭐⭐: XOR all elements — pairs cancel, leaving the unique one. Matters as the foundational XOR-cancellation trick, extended in #183/#184. Prereq: none. Approach: `result ^= num` for all nums. Insight: XOR is commutative/associative, so order doesn't matter — pairs annihilate regardless of arrangement. Mistake: reaching for a hash set (works, O(n) space) instead of the O(1)-space XOR trick. Companies: Amazon, Palantir.

#178. Number of 1 Bits — Easy · 🟢 · ⭐⭐⭐⭐: Count set bits via `n & (n-1)` repeatedly (clears lowest set bit each time) or shift-and-check. Matters as the introduction to the `n & (n-1)` trick, reused in #179 and power-of-two checks. Prereq: none. Approach: loop while `n!=0`, `count++; n &= n-1`. Insight: this loop runs exactly as many times as there are set bits — not 32 times — making it slightly more efficient than a full shift-scan. Mistake: using string conversion (`bin(n).count('1')`) — works but misses the intended bit-trick practice. Companies: Apple, Microsoft.

#179. Counting Bits — Easy | 🔵 | ⭐⭐⭐⭐: DP using the relation `dp[i] = dp[i >> 1] + (i & 1)`. Matters as a bit-manipulation problem elevated by a DP insight — each number's bit count relates to a smaller previously-computed number's. Prereq: #178. Approach: bottom-up array, `dp[i] = dp[i//2] + i%2`. Insight: recognizing `i>>1` (integer division by 2) as "drop the last bit" is what connects this to a previously solved smaller subproblem. Mistake: recomputing bit count from scratch for every number (correct but misses the intended O(n) DP relation, ending up at O(n log n) instead). Companies: Google.

#180. Reverse Bits — Easy · 🟢 · ⭐⭐⭐: Bit-by-bit extraction and reconstruction in reverse order. Matters as a straightforward bit-manipulation mechanics exercise. Prereq: none. Approach: loop 32 times, shift result left, OR in the lowest bit of n, shift n right. Insight: processing exactly 32 iterations (fixed-width integer assumption) is important — don't loop based on `n`'s current value. Mistake: looping while `n!=0`, which stops early and produces a wrong-length reversed result. Companies: Apple.

#181. Missing Number — Easy · 🟢 · ⭐ ⭐ ⭐ ⭐: XOR indices with values (or sum formula `n(n+1)/2 - sum(nums)`). Matters as another XOR-cancellation application, and a good contrast against the simpler arithmetic-sum alternative. Prereq: #177. Approach (XOR): XOR all indices `0..n` and all values together — everything but the missing number cancels. Approach (sum): expected sum minus actual sum. Insight: the XOR approach avoids potential integer-overflow concerns that the sum-formula approach could face on very large inputs in other languages. Mistake: off-by-one when ranging indices `0..n` inclusive. Companies: Microsoft, Amazon.

#182. Sum of Two Integers — Medium · 🔴 · ⭐ ⭐ ⭐: Add two integers using only bitwise operations — XOR gives sum without carry, AND-then-shift gives the carry, repeat until no carry remains. Matters as a deep-dive into how addition actually works at the bit level, a favorite "no arithmetic operators allowed" constraint question. Prereq: #177. Approach: loop `while carry != 0: carry = (a&b)<<1; a = a^b; b = carry`. Insight: this loop is literally simulating how a hardware adder works internally — XOR is the "sum bit," AND+shift is the "carry bit." Mistake: not handling negative numbers correctly in languages without fixed-width integer wraparound (e.g., Python needs manual masking). Companies: Google.

#183. Single Number II — Medium · 🔴 · ⭐ ⭐ ⭐: Bit-counting mod 3 across all numbers (every bit of the answer is the one bit-position where the total count isn't a multiple of 3), or a state-machine (ones/twos) bitwise trick. Matters as a deeper generalization of #177 to "every element appears 3 times except one." Prereq: #177. Approach: for each of the 32 bit positions, sum that bit across all numbers mod 3; set that bit in the answer if the remainder is 1. Insight: the "count each bit position independently mod k" generalization is the transferable insight extending far beyond just k=3. Mistake: attempting to directly generalize the simple XOR trick from #177, which does not work when elements repeat 3 times (XOR of three identical values isn't 0). Companies: Google.

#184. Single Number III — Medium · 🔴 · ⭐ ⭐ ⭐: XOR everything to get `a^b` (the two unique numbers XORed), find any set bit in that result to partition all numbers into two groups, then XOR within each group. Matters as combining #177's trick with a clever partitioning step to separate two unknowns. Prereq: #177. Approach: full XOR gives `a^b`; isolate lowest set bit (`diff & -diff`); partition all numbers by whether that bit is set; XOR each partition separately to isolate `a` and `b`. Insight: any bit where `a` and `b` differ (found via `a^b`) can be used to correctly separate every other paired-up number into the same partition as its duplicate. Mistake: not realizing you need to isolate just *one* differing bit, not the entire `a^b` value, for correct partitioning. Companies: Meta.

#185. Subsets — (see #76, revisited here as a bitmask-enumeration problem: each integer `0..2^n-1` represents a unique subset via its binary representation.) Insight for this lens: iterating `mask` from `0` to `2^n - 1` and checking each bit gives an alternative iterative subset-generation approach to backtracking. Companies: Amazon, Google.

---

16. Math & Number Theory

Pattern Card

- **Recognition signals:** primes, GCD/LCM, modular arithmetic ("answer modulo 10^9+7"), combinatorics counting, "power" computation, digit manipulation.
- **Core idea:** most interview math reduces to a handful of reusable tools — Euclidean algorithm for GCD, Sieve of Eratosthenes for prime enumeration, fast exponentiation for large powers,

and modular arithmetic identities to keep numbers bounded.

- **Typical complexity:** O(log(min(a,b))) for GCD, O(n log log n) for sieve, O(log n) for fast exponentiation.
- **Common traps:** integer overflow (mitigated by modular arithmetic); forgetting `1` is not prime and `2` is the only even prime; sieve implemented with an O(n²) inner loop instead of proper O(n log log n) marking.

#	Problem	Difficulty	Level	Priority
186	Greatest Common Divisor of Strings	Easy	🟢	⭐ ⭐ ⭐
187	Count Primes	Medium	🔵	⭐ ⭐ ⭐ ⭐
188	Pow(x, n)	Medium	🔵	⭐ ⭐ ⭐ ⭐
189	Happy Number	Easy	🟢	⭐ ⭐ ⭐
190	Roman to Integer	Easy	🟢	⭐ ⭐ ⭐
191	Factorial Trailing Zeroes	Medium	🔴	⭐ ⭐
192	Excel Sheet Column Number	Easy	🟢	⭐ ⭐
193	Random Pick with Weight	Medium	🔴	⭐ ⭐ ⭐

Problem Details

#186. Greatest Common Divisor of Strings — Easy · 🟢 · ⭐ ⭐ ⭐: Euclidean-algorithm analogy applied to strings. Key concept: two strings have a valid "GCD string" iff `str1+str2 == str2+str1`; the answer's length is `gcd(len1, len2)`. Matters as a creative application of GCD reasoning beyond plain numbers. Prereq: know Euclid's algorithm. Approach: check the concatenation-equality condition, then slice the prefix of length `gcd(len1,len2)`. Insight: the concatenation check is a clean way to verify both strings are built from repetitions of a common base string. Mistake: trying to brute-force search for the GCD string by trying all prefix lengths instead of the direct gcd-length computation. Companies: Google.

#187. Count Primes — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Sieve of Eratosthenes. Key concept: mark multiples of each prime starting from `p*p` (smaller multiples already marked by smaller primes). Matters as the canonical prime-enumeration algorithm, essential general knowledge for math-flavored interview problems. Prereq: none. Approach: boolean array up to n, iterate `p` from 2 to sqrt(n), if unmarked mark all multiples of `p` starting at `p*p`. Insight: starting marking from `p*p` instead of `2*p` is the key optimization that gives the true O(n log log n) complexity. Mistake: starting the inner marking loop from `2*p`, which is still correct but slightly less optimal, and not knowing why `p*p` is the right starting point. Companies: Bloomberg, Adobe.

#188. Pow(x, n) — Medium · 🔵 · ⭐ ⭐ ⭐ ⭐: Fast exponentiation by repeated squaring — `x^n = (x^(n/2))^2`, adjusted for odd n. Matters as the canonical O(log n) exponentiation technique, applicable anywhere large powers are needed (matrix exponentiation, modular exponentiation in

cryptography-adjacent problems). Prereq: recursion basics. Approach: recursive or iterative halving of the exponent, squaring the base each step. Insight: handling negative exponents (`x^-n = 1/x^n`) and the edge case `n=INT_MIN` (can't simply negate) are the easy-to-miss details. Mistake: naive O(n) repeated multiplication instead of O(log n) fast exponentiation. Companies: Meta, LinkedIn.

#189. Happy Number — Easy · 🟢 · ⭐ ⭐ ⭐: Cycle detection (Floyd's, #48) applied to a sequence of digit-square-sums instead of a linked list. Matters as a great example of recognizing "this sequence-with-possible-cycle problem is really Floyd's algorithm in disguise," reinforcing pattern transfer across topics. Prereq: #48. Approach: apply fast/slow pointer to the sequence generated by `n -> sum of squares of digits`; if it reaches 1, happy; if it cycles without reaching 1, not happy. Insight: any deterministic sequence that might loop forever (not just literal linked lists) can be cycle-detected with Floyd's technique — a hash set also works but uses O(n) space. Mistake: not recognizing the cycle-detection connection and writing an unbounded loop with no termination guarantee. Companies: Amazon.

#190. Roman to Integer — Easy · 🟢 · ⭐ ⭐ ⭐: Left-to-right scan with a "subtract if smaller-before-larger" rule. Matters as a simple parsing/math hybrid, common as a warm-up question. Prereq: none. Approach: map each numeral to its value; add each value, but subtract twice the previous value if the previous numeral was smaller than the current one (undoing the wrong earlier addition). Insight: processing right-to-left and comparing each value to the running max avoids needing the "subtract twice" correction entirely — a cleaner formulation worth knowing. Mistake: forgetting to double-subtract (once to cancel the earlier addition, once to actually subtract) when scanning left-to-right. Companies: Meta, Uber.

#191. Factorial Trailing Zeroes — Medium · 🟠 · ⭐ ⭐: Count factors of 5 in `n!`, since factors of 2 always outnumber factors of 5. Key concept: `trailingZeroes(n) = n/5 + n/25 + n/125 + ...` Matters as a neat number-theory shortcut avoiding computing the actual (enormous) factorial. Prereq: basic number theory. Approach: sum `n // 5^k` for increasing k until it's 0. Insight: recognizing that trailing zeroes come from factor pairs of (2,5), and 2s are always more abundant than 5s, reduces the problem to "count factors of 5 only." Mistake: trying to actually compute `n!` for large n (overflow/impossibly slow). Companies: Google.

#192. Excel Sheet Column Number — Easy · 🟢 · ⭐ ⭐: Base-26 conversion, but with a twist — it's a "bijective base-26" system with no true zero digit. Matters as a numeral-system-conversion exercise with a subtle off-by-one twist compared to standard base conversion. Prereq: none. Approach: `result = result*26 + (char - 'A' + 1)` for each character. Insight: unlike standard base-26 (digits 0-25), Excel columns use digits 1-26 with no zero — this is why the formula uses `+1` and doesn't need special-casing despite initially looking like standard base conversion. Mistake: treating it as standard base-26 with a 0-25 digit range, causing subtly wrong answers on multi-letter columns. Companies: Google, Facebook.

#193. Random Pick with Weight — Medium · 🟠 · ⭐ ⭐ ⭐: Prefix sums + binary search for weighted random sampling. Key concept: build a prefix-sum array of weights, generate a uniform random number in `[1, totalWeight]`, binary search for the first prefix sum ⩾ that number. Matters as a nice combination of prefix sums (#13) and binary search (#36) applied to a probability/sampling context. Prereq: #13, #36. Approach: precompute prefix sums once; each `pickIndex()` call does one random number generation + one binary search. Insight: the

probability of landing in bucket `i` is proportional to `weight[i]` precisely because it occupies that proportional a range of the prefix-sum number line. Mistake: linear-scanning for the bucket instead of binary searching (correct but slower per query). Companies: Google, Amazon.

---

17. Tries

Pattern Card

- **Recognition signals:** "prefix," "autocomplete," "word dictionary with wildcard search," "longest common prefix of many strings," XOR-maximization over a set of numbers.
- **Core idea:** a trie (prefix tree) shares common prefixes across many strings in a tree structure, giving O(L) insert/search (L = string length) regardless of how many strings are stored — far better than storing all strings in a set for prefix-based queries.
- **Typical complexity:** O(L) per insert/search where L is the word length; O(N·L) to build from N words.
- **Common traps:** forgetting to mark "end of word" nodes (causing false-positive prefix matches to register as full-word matches); not handling wildcard characters (`.`) which require trying all children at that level (effectively DFS through the trie).

#	Problem	Difficulty	Level	Priority
194	Implement Trie (Prefix Tree)	Medium	🔵	⭐⭐⭐⭐⭐
195	Design Add and Search Words Data Structure	Medium	🟠	⭐⭐⭐⭐
196	Word Search II	Hard	🔴	⭐⭐⭐⭐⭐
197	Maximum XOR of Two Numbers in an Array	Medium	🔴	⭐⭐⭐
198	Longest Word in Dictionary	Medium	🟠	⭐⭐

Problem Details

#194. Implement Trie (Prefix Tree) — Medium · 🔵 · ⭐⭐⭐⭐⭐: Build the core trie data structure from scratch. Key concept: each node holds a map/array of children keyed by character, plus an `isEndOfWord` flag. Matters as the mandatory foundational trie implementation every other trie problem builds on. Prereq: tree/recursion basics. Approach: `insert` walks/creates nodes per character, marking the last as end-of-word; `search` walks and checks the end flag; `startsWith` walks without checking the flag. Insight: using a fixed-size array of 26 children (for lowercase-only inputs) is faster than a hash map, an important optimization to mention. Mistake: forgetting the `isEndOfWord` flag, causing `search("app")` to incorrectly return true when only "apple" was inserted. Companies: Google, Amazon, Meta — a mandatory building block.

#195. Design Add and Search Words Data Structure — Medium · 🟠 · ⭐⭐⭐⭐: Trie + DFS to support wildcard (`.`) search. Key concept: on encountering `.`, recursively try every child at that level instead of following a single path. Matters as the natural extension of #194 to support pattern matching. Prereq: #194. Approach: `search` recurses with an index into the word; a `.` branches into all current node's children. Insight: this is a DFS over the trie's structure guided by the query

string, branching only at wildcard positions. Mistake: not handling multiple consecutive wildcards efficiently, potentially causing excessive branching if the trie is large and poorly pruned (generally still acceptable for typical constraints). Companies: Meta.

#196. Word Search II — Hard · 🔴 · ⭐ ⭐ ⭐ ⭐ ⭐: Trie of all target words + grid backtracking (#83), pruning branches that don't match any trie path. Key concept: instead of running #83's Word Search once per word (slow for many words), build one trie of all words and do a single combined DFS over the grid, following trie edges. Matters as one of the best "combine two known techniques for a multiplicative speedup" hard problems — trie pruning turns an otherwise infeasible multi-word search into a tractable one. Prereq: #194, #83. Approach: build a trie from the word list; DFS from every grid cell, walking both the grid and the trie simultaneously, marking found words and removing them from the trie to avoid duplicate detection. Insight: pruning the trie by removing leaf nodes after a word is found (or once a path yields no viable children) meaningfully speeds up long searches over large word lists. Mistake: running backtracking per word independently (correct but far too slow with many words — O(words × grid backtracking) instead of O(grid backtracking) once). Companies: Amazon, Google, Meta — a favorite hard-tier combination question.

#197. Maximum XOR of Two Numbers in an Array — Medium · 🔴 · ⭐ ⭐ ⭐: Binary trie over bit representations, greedily choosing the opposite bit at each level to maximize XOR. Key concept: insert every number's binary representation (MSB to LSB) into a trie; for each number, greedily walk the trie preferring the *opposite* bit at each position to maximize the XOR result. Matters as a clever, less obvious application of tries beyond strings — bitwise tries for numeric maximization problems. Prereq: #194, #182 (bit manipulation fluency). Approach: build trie of fixed-length (e.g., 32-bit) binary representations; for each number, travert the trie trying to go opposite-bit whenever possible, accumulate the resulting max XOR candidate. Insight: recognizing that "maximize XOR" is equivalent to "greedily differ at the most significant bit possible" is the leap that connects XOR maximization to trie traversal. Mistake: brute-forcing all $O(n^2)$ pairs instead of the $O(n \cdot 32)$ trie approach. Companies: Google.

#198. Longest Word in Dictionary — Medium · 🟠 · ⭐ ⭐: Trie where a word is "buildable" only if every prefix of it is also a complete word in the dictionary. Key concept: insert all words into a trie, then DFS only through nodes marked as end-of-word, tracking the longest (lexicographically smallest on ties) path. Matters as a good "trie + traversal-with-constraint" exercise. Prereq: #194. Approach: build trie; DFS from root, only continuing into children that are themselves marked end-of-word. Insight: the "each prefix must also be a valid word" constraint maps directly onto "only traverse trie edges leading to end-of-word nodes" — a clean structural translation. Mistake: checking word validity via repeated dictionary/set lookups per prefix (works but $O(L^2)$ per word) instead of the $O(L)$ trie-constrained DFS. Companies: Google.

---

18. Advanced Data Structures

Pattern Card

- **Recognition signals:** "range sum/min/max query with updates" (mutable range queries), "count of smaller elements after self," "k-th order statistic with updates," anything requiring both point updates AND range queries efficiently.

- **Core idea:** plain prefix sums/arrays are O(1) query but O(n) update; a Segment Tree or Fenwick Tree (Binary Indexed Tree) achieves O(log n) for *both* point updates and range queries — the standard structure whenever both are required together.
- **Typical complexity:** O(log n) per update/query; O(n) or O(n log n) to build.
- **Common traps:** off-by-one indexing (Fenwick Trees are conventionally 1-indexed); forgetting a segment tree needs `4n` space for a straightforward array-based implementation; over-engineering with these structures when a simpler prefix-sum/sort suffices (immutable range queries never need them).

#	Problem	Difficulty	Level	Priority
199	Range Sum Query - Mutable	Medium	🟠	⭐ ⭐ ⭐ ⭐
200	Count of Smaller Numbers After Self	Hard	🔴	⭐ ⭐ ⭐
201	The Skyline Problem	Hard	🔴	⭐ ⭐ ⭐
202	Design a Leaderboard	Medium	🟠	⭐ ⭐
203	My Calendar I	Medium	🟠	⭐ ⭐

Problem Details

#199. Range Sum Query - Mutable — Medium · 🟠 · ⭐ ⭐ ⭐ ⭐: Fenwick Tree (BIT) or Segment Tree supporting O(log n) point update and O(log n) range sum query. Matters as the definitive introduction to why plain prefix sums (#13) fail once updates are introduced, motivating these advanced structures. Prereq: #13. Approach (Fenwick): array indexed by `i & -i` (lowest set bit) jumps for both update and prefix-sum query. Insight: the contrast with #13 is the whole point — a mutable array needs a fundamentally different structure to keep both operations logarithmic; a naive prefix-sum array is O(1) query but O(n) update, which fails under frequent updates. Mistake: recomputing prefix sums from scratch after every update (O(n) per update, too slow for many updates). Companies: Google, Amazon.

#200. Count of Smaller Numbers After Self — Hard · 🔴 · ⭐ ⭐ ⭐: Fenwick Tree over compressed value ranks, processing the array right-to-left (or merge-sort based counting). Key concept: for each element, count how many smaller elements have been "inserted" into the Fenwick tree so far (processing from the end backward). Matters as a genuinely hard application combining coordinate compression with a Fenwick Tree, or alternatively solvable via modified merge sort — both are valuable to know. Prereq: #199. Approach: compress values to ranks, iterate right-to-left, query count of ranks less than current, then insert current rank. Insight: coordinate compression (mapping values to their sorted rank) is essential since raw values may be too large/sparse for a direct Fenwick array index. Mistake: forgetting to compress coordinates first, causing infeasible array sizes for large value ranges. Companies: Google.

#201. The Skyline Problem — Hard · 🔴 · ⭐ ⭐ ⭐: Sweep-line + a max-heap (or a balanced multiset/ordered structure) tracking currently active building heights. Key concept: process building edges (start=add height, end=remove height) in sorted x-order, tracking the current max height at each event, outputting a new skyline point whenever the max height changes. Matters as

an iconic hard problem combining sweep-line thinking with a heap or ordered-structure for tracking "currently active maximum." Prereq: #175 (sweep-line from Meeting Rooms II), heap fluency. Approach: create `(x, height, isStart)` events, sort them (careful tie-breaking for simultaneous starts/ends), sweep through maintaining a max-heap (with lazy deletion for expired buildings) or a multiset of active heights. Insight: correctly breaking ties among simultaneous events (process all starts at an x before ends, and taller starts before shorter ones) is the trickiest correctness detail. Mistake: incorrect event ordering/tie-breaking, producing skylines with spurious extra points. Companies: Google, Meta, Microsoft — a legendarily tricky hard problem.

#202. Design a Leaderboard — Medium · 🔴 · ⭐ ⭐: Hash map for scores + sorting (or a Fenwick Tree/ordered structure) for top-K queries. Matters as a practical design problem testing whether advanced structures are even necessary (often a hash map + sort suffices at LeetCode's given scale). Prereq: #4 (hashing), optionally #199. Approach: hash map of player→score; `top(K)` sorts current scores and sums the top K (acceptable given constraints); a Fenwick Tree over score ranks would optimize this further for very frequent queries. Insight: recognizing when a "fancy" advanced structure is overkill relative to given constraints — and being able to articulate the trade-off — is itself a valued interview skill. Mistake: over-engineering with a full Fenwick Tree/segment tree when the given constraints make simple sorting perfectly adequate. Companies: Amazon.

#203. My Calendar I — Medium · 🔴 · ⭐ ⭐: Store booked intervals and linearly (or via a sorted structure/segment tree) check for overlap on each new booking. Matters as an entry point into "online interval insertion with overlap checking," extendable to segment-tree-based solutions for the harder follow-ups (My Calendar II/III use exactly this same skeleton with added complexity). Prereq: #171 (interval overlap logic). Approach: maintain a sorted list (or simple list, fine at LeetCode's scale) of booked intervals; check `newStart < existingEnd and existingStart < newEnd` against all others before inserting. Insight: the follow-up problems (double/triple booking) naturally motivate moving from a naive O(n) check per booking to a segment tree over compressed time ranges — good to mention proactively. Mistake: not recognizing this simple version as a stepping stone toward the segment-tree-required harder variants. Companies: Google.

---

19. Learning Tracks

> **Note on scope:** This guide curates **203 distinct problems** (not 300) deliberately — every problem earns its place by teaching a *distinct* pattern or a meaningfully different variant. Two problems that would teach the same lesson twice were merged into one entry with cross-references instead (e.g., Maximum Subarray appears once, referenced from both Arrays & Hashing and Greedy). This keeps solve-time proportional to learning, not to a headline number.

Track A — Foundation

For someone comfortable with basic programming but still building problem-solving intuition. Goal: get fluent with every core pattern at the 🟢/🔵 level before touching 🟠/🔴.

Focus exclusively on 🟢 Foundation and 🔵 Pattern Building problems across every topic — roughly **95 problems**. Do NOT attempt Hard problems yet. Priority order: Arrays & Hashing → Two Pointers → Sliding Window → Binary Search → Linked Lists → Stack → Recursion/Backtracking (subsets/permutations only) → Trees (traversal-level) → Intervals (merge only) → Greedy (Jump

Game family). Skip Graphs, DP, Tries, and Advanced Data Structures entirely at this stage — return to them once Track A topics feel automatic.

Track B — Placement

The minimum high-value set needed for strong general software-engineering placement preparation (most product/service companies, not necessarily MAANG-tier bars). Goal: broad pattern coverage with solid depth on mediums, light exposure to hards.

All ⭐ ⭐ ⭐ ⭐ and ⭐ ⭐ ⭐ ⭐ ⭐ problems across every topic (roughly **150 problems**), including the essential Graphs (Number of Islands, Clone Graph, Course Schedule), core DP (Climbing Stairs through Longest Common Subsequence, Coin Change, Word Break, LIS, Partition Equal Subset Sum), and one pass through Tries (#194, #195). Hards are optional here except the handful marked ⭐ ⭐ ⭐ ⭐ ⭐ (Trapping Rain Water, Median of Two Sorted Arrays, Merge k Sorted Lists, Word Ladder, Edit Distance, Serialize/Deserialize Binary Tree) — these are asked often enough to be worth the stretch.

Track C — MAANG

The complete, challenging set for strong product-company interviews. Goal: fluency across all patterns including hards, and the ability to derive a solution to an unseen variant by recognizing which pattern(s) it combines.

The entire 203-problem set, with particular emphasis on the 🔴 MAANG-level and Advanced Data Structures sections (Segment/Fenwick Trees, Skyline Problem, Word Search II) which differentiate strong candidates. Also revisit every "Follow-up / Variation" mentioned in the Problem Details — MAANG interviewers frequently ask the base problem then pivot to the variation live.

Highest-ROI problems (show up across the most companies, and each teaches a pattern reused in 5+ other problems): Two Sum, 3Sum, Longest Substring Without Repeating Characters, Product of Array Except Self, Trapping Rain Water, Merge Intervals, Number of Islands, Course Schedule, Clone Graph, Word Break, Longest Common Subsequence, Coin Change, Kth Largest Element in an Array, LRU Cache, Merge k Sorted Lists, Binary Tree Maximum Path Sum, Validate Binary Search Tree, Word Search II. If time is critically short, these ~18 alone give the broadest pattern coverage per problem solved.

---

20. Revision System

20.1 — 50 Essential Problems (solve almost from memory)

Two Sum · Contains Duplicate · Group Anagrams · Top K Frequent Elements · Product of Array Except Self · Longest Consecutive Sequence · Maximum Subarray · 3Sum · Container With Most Water · Trapping Rain Water · Best Time to Buy and Sell Stock · Longest Substring Without Repeating Characters · Longest Repeating Character Replacement · Minimum Window Substring · Binary Search · Search in Rotated Sorted Array · Koko Eating Bananas · Reverse Linked List · Merge Two Sorted Lists · Linked List Cycle · Remove Nth Node From End of List · LRU Cache · Merge k Sorted Lists · Valid Parentheses · Daily Temperatures · Largest Rectangle in Histogram · Sliding Window Maximum · Task Scheduler · Binary Tree Level Order Traversal · Subsets · Permutations · Combination Sum · Word Search · Generate Parentheses · Invert Binary Tree ·

Maximum Depth of Binary Tree · Diameter of Binary Tree · Lowest Common Ancestor of a Binary Tree · Validate Binary Search Tree · Construct Binary Tree from Preorder and Inorder Traversal · Binary Tree Maximum Path Sum · Kth Largest Element in an Array · Find Median from Data Stream · Number of Islands · Clone Graph · Course Schedule · Word Ladder · Climbing Stairs · House Robber · Coin Change · Longest Common Subsequence

20.2 — 100 Core Problems (most important to repeatedly revise)

All 50 Essential problems above, plus: Valid Anagram · Subarray Sum Equals K · Maximum Product Subarray · First Missing Positive · Sort Colors · 3Sum Closest · Boats to Save People · Permutation in String · Fruit Into Baskets · Find First and Last Position of Element in Sorted Array · Find Minimum in Rotated Sorted Array · Median of Two Sorted Arrays · Reorder List · Copy List with Random Pointer · Add Two Numbers · Reverse Nodes in k-Group · Min Stack · Evaluate Reverse Polish Notation · Car Fleet · Decode String · Rotting Oranges · Design Circular Queue · Subsets II · Word Search II · Palindrome Partitioning · N-Queens · Letter Combinations of a Phone Number · Same Tree · Balanced Binary Tree · Kth Smallest Element in a BST · Serialize and Deserialize Binary Tree · Binary Tree Right Side View · K Closest Points to Origin · Reorganize String · Number of Connected Components in an Undirected Graph · Course Schedule II · Is Graph Bipartite? · Network Delay Time · Cheapest Flights Within K Stops · Alien Dictionary · Min Cost to Connect All Points · House Robber II · Decode Ways · Word Break · Longest Increasing Subsequence · Partition Equal Subset Sum · Unique Paths · Edit Distance · Longest Palindromic Substring · Best Time to Buy and Sell Stock with Cooldown · Burst Balloons · Jump Game · Jump Game II · Gas Station · Partition Labels · Non-overlapping Intervals · Merge Intervals · Insert Interval · Meeting Rooms II · Single Number · Number of 1 Bits · Missing Number · Implement Trie (Prefix Tree) · Design Add and Search Words Data Structure

20.3 — Pattern Revision Sheet

Pattern	Recognize It By	Standard Approach	Time Complexity	Typical Structure	Representative Problems
Hashing	frequency/lookup, "have I seen this," grouping	hash map/set, one or two passes	O(n)	HashMap / HashSet	Two Sum, Group Anagrams, Subarray Sum Equals K
Two Pointers	sorted array, pair/triplet sum, palindrome	opposite or same-direction pointers	O(n) or O(n log n) w/ sort	two indices	3Sum, Container With Most Water, Sort Colors
Sliding Window	contiguous subarray/substring, longest/shortest w/ constraint	expand right, shrink left on violation	O(n)	two pointers + map/counter	Longest Substring w/o Repeat, Minimum Window Substring
Binary Search	sorted data, or "minimize/maximize feasible X"	halve search space; on-answer via feasibility fn	O(log n) or O(n log range)	index bounds lo/hi	Search Rotated Array, Koko Eating Bananas
Linked List Pointers	reverse/cycle/middle/merge on ListNode	prev/curr/next; fast/slow; dummy head	O(n), O(1) space	prev, curr, fast, slow	Reverse Linked List, LL Cycle, Merge k Sorted Lists
Monotonic Stack	next greater/smaller, histogram	push while increasing/decreasing, resolve on violation	O(n)	stack of indices	Daily Temperatures, Largest Rectangle in Histogram
Monotonic Queue	sliding window max/min	deque evicting from both ends	O(n)	deque of indices	Sliding Window Maximum
Backtracking	generate all subsets/perms/combos w/ constraints	choose → recurse → undo	O(2^n)/O(n!)	recursion + path list	Subsets, Permutations, N-Queens

Pattern	Recognize It By	Standard Approach	Time Complexity	Typical Structure	Representative Problems
Tree DFS/BFS	TreeNode, height/path/LCA/traversal	recursive return-up or state-pass-down; BFS queue for levels	O(n)	recursion / queue	Diameter, LCA, Level Order Traversal
Heap / Top-K	Kth largest, top K, running median, merge K	bounded heap (size k), or two heaps for median	O(n log k)	min/max heap	Kth Largest Element, Find Median from Data Stream
Graph BFS/DFS	adjacency list/grid, reachability, shortest path (unweighted)	visited set + queue (BFS) or recursion (DFS)	O(V+E)	queue / recursion + visited	Number of Islands, Clone Graph, Word Ladder
Topological Sort	prerequisite/dependency ordering	Kahn's BFS (indegree) or DFS post-order	O(V+E)	indegree array + queue	Course Schedule, Alien Dictionary
Union-Find	dynamic connectivity, cycle detection (undirected)	union by rank + path compression	~O(E)	parent/rank arrays	Redundant Connection, Number of Connected Components
Dijkstra / Weighted Shortest Path	weighted graph, shortest/cheapest path	min-heap + relaxation	O(E log V)	heap of (dist, node)	Network Delay Time, Cheapest Flights
1D DP	take/skip, "ways to reach step i"	dp[i] from dp[i-1], dp[i-2]...	O(n)	array or rolling vars	Climbing Stairs, House Robber, Coin Change
2D / String DP	two strings/sequences, edit/match/common	dp[i][j] from neighbors	O(n·m)	2D table	LCS, Edit Distance, Longest Palindromic Substring
Knapsack (0/1 vs unbounded)	subset sum/count with item constraint	reuse: loop item outer; once: sum descending	O(n·target)	1D/2D dp array	Partition Equal Subset Sum, Coin Change
Interval	[start,end] pairs, merge/overlap/rooms	sort by start or end, sweep	O(n log n)	sorted array / heap	Merge Intervals,

Pattern	Recognize It By	Standard Approach	Time Complexity	Typical Structure	Representative Problems
					Meeting Rooms II
Greedy	locally optimal choice, provable exchange argument	sort, then single pass making the obvious choice	O(n log n)	sorted array	Jump Game, Non-overlapping Intervals
Bit Manipulation	XOR cancellation, set bits, powers of two	XOR trick, n&(n-1), bitmask	O(1)–O(32)	int / bitmask	Single Number, Counting Bits
Trie	prefix search, wildcard search, word dictionary	tree of chars with end-of-word flags	O(L) per op	trie node w/ children array	Implement Trie, Word Search II
Fenwick / Segment Tree	mutable range sum/min/max query	tree structure with O(log n) update+query	O(log n)	BIT array / segment array	Range Sum Query - Mutable, Count of Smaller Numbers

20.4 — Mistake Log Template

Keep this running in a spreadsheet or notes app; review it weekly.

Problem	Mistake	Why I Made It	Correct Insight	Pattern	How To Avoid It
(e.g., Coin Change)	*(e.g., forgot infinity init)*	*(e.g., assumed 0 was a safe default)*	*(e.g., unreachable amounts must start at infinity)*	*(e.g., Unbounded Knapsack)*	*(e.g., always ask "what does an unreachable state look like?" before coding)*

20.5 — Interview Simulation Sets

Each set mixes topics the way a real 45–60 minute interview loop would. Time yourself: ~20 minutes per medium, ~30–35 for a hard, with no looking up solutions.

- **Simulation 1 (Easy warm-up + 2 Medium):** Two Sum → Group Anagrams → Non-overlapping Intervals
- **Simulation 2 (Array/String heavy):** Longest Substring Without Repeating Characters → 3Sum → Longest Palindromic Substring
- **Simulation 3 (Tree + Graph):** Validate Binary Search Tree → Number of Islands → Course Schedule

- **Simulation 4 (Linked List + Design):** Reverse Linked List → LRU Cache → Merge k Sorted Lists
- **Simulation 5 (DP focus):** House Robber → Coin Change → Longest Common Subsequence
- **Simulation 6 (Hard-heavy, MAANG bar):** Trapping Rain Water → Word Break → Median of Two Sorted Arrays
- **Simulation 7 (Backtracking + Greedy):** Subsets → Combination Sum → Jump Game II
- **Simulation 8 (Heap + Graph, senior-leaning):** Find Median from Data Stream → Network Delay Time → Word Ladder
- **Simulation 9 (Full MAANG hard loop):** Serialize and Deserialize Binary Tree → Alien Dictionary → Edit Distance
- **Simulation 10 (Mixed grab-bag, final check):** Product of Array Except Self → Meeting Rooms II → Binary Tree Maximum Path Sum

---

21. 12–16 Week Preparation Roadmap

Assumes ~1.5–2.5 hours/day. Adjust pace based on your actual available time — better to stretch to 16 weeks and retain everything than rush 12 weeks and forget half.

Week	Topics	Problems (approx.)	Revision	Mock Interviews
1	Arrays & Hashing	#1–16	—	—
2	Two Pointers, Sliding Window	#17–34	Re-solve 3 hardest from Week 1	—
3	Binary Search, Linked Lists	#35–57	Re-solve 3 hardest from Week 2	—
4	Stack & Monotonic Stack, Queue	#58–75	Weeks 1–3 spot check (5 random)	Simulation 1
5	Recursion & Backtracking	#76–88	Re-solve 3 hardest from Week 4	—
6	Trees (part 1: traversal, BST)	#89–100	Weeks 1–5 spot check	Simulation 3 (partial)
7	Trees (part 2: construction, hard)	#101–104 + catch-up	Re-solve 3 hardest from Week 6	—
8	Heap / Priority Queue	#105–114	Weeks 1–7 spot check	Simulation 4
9	Graphs (part 1: BFS/DFS, Union-Find)	#115–126	Re-solve 3 hardest from Week 8	—
10	Graphs (part 2: topo sort, Dijkstra, hard)	#127–133	Weeks 1–9 spot check	Simulation 3 (full), Simulation 8
11	Dynamic Programming (part 1: 1D, knapsack)	#134–148	Re-solve 3 hardest from Week 10	—
12	Dynamic Programming (part 2: 2D, hard)	#149–160	Weeks 1–11 spot check	Simulation 5, Simulation 6
13	Greedy, Intervals	#161–176	Re-solve 3 hardest from Week 12	Simulation 7
14	Bit Manipulation, Math, Tries	#177–198	Weeks 1–13 spot check	Simulation 9
15	Advanced Data Structures + full repeat of 50 Essential	#199–203 + Essentials	Full 50 Essential re-solve	Simulation 2, Simulation 10
16	Mock-interview week: 2 simulations/day, weakest-topic re-drill	—	Targeted re-solve of every Mistake Log entry	2 full mock loops/day

If compressing to 12 weeks: merge weeks 6–7, 9–10, and 11–12 pairs into single weeks each, and drop Track C-only content (Advanced Data Structures, several 🔴 hards) unless targeting a MAANG-specific bar.

Daily Workflow

1. **Learn the pattern** (10–15 min) — read that topic's Pattern Card before touching any problem.
2. **Solve 2–3 problems** (60–90 min) — work in increasing difficulty order within the day's topic; timebox each attempt (20 min medium / 30 min hard) before checking a hint.
3. **Review mistakes** (10 min) — log anything you got wrong or solved inefficiently into the Mistake Log (§20.4) immediately, while it's fresh.
4. **Re-solve one difficult problem from a prior session** (15–20 min) — pick the oldest unresolved Mistake Log entry, not the most recent.
5. **Spaced revision** (5–10 min) — once a week, re-attempt 3–5 problems from the 50 Essential list (§20.1) cold, no notes, to confirm true retention.

End of guide. Revisit the Pattern Cards before every topic, keep the Mistake Log honest, and prioritize the ⭐ ⭐ ⭐ ⭐ ⭐ problems whenever time is short.